



KLE Technological University

Creating Value,
Leveraging Knowledge

A Minor Project report on

Intra-CU Inter-DU Handover Simulation in 5G SA CU-DU Split Architecture using OpenAirInterface5G

Submitted

in partial fulfillment of the requirements for the award of the degree of

Bachelor of Engineering

IN

Department of Computer Science & Engineering

Submitted By

Harsha Undodi	01FE22BCS300
Anirudh P Kulkarni	01FE22BCS049
Team Member 3	USN
Team Member 4	USN

Under the guidance of

Guide Name

Designation

Department of Computer Science and Engineering

KLE Technological University, Hubballi

2025-2026

ABSTRACT

With the evolution of Fifth Generation (5G) cellular networks, the 3rd Generation Partnership Project (3GPP) introduced the Centralized Unit (CU) and Distributed Unit (DU) functional split in the Next Generation Radio Access Network (NG-RAN). This split, formally known as the 3GPP Option 2 Higher Layer Split (HLS), separates the control-plane and user-plane processing between a Centralized Unit handling RRC, SDAP, and PDCP layers, and one or more Distributed Units handling RLC, MAC, and PHY layers. The standardized F1 interface interconnects these entities, enabling flexible deployment, resource pooling, and efficient mobility management.

One of the critical mobility scenarios in this architecture is the **intra-CU inter-DU handover** (F1 handover), which allows a User Equipment (UE) to switch its serving DU under a common CU without involving the 5G Core control plane. This project implements and simulates an end-to-end 5G Standalone (SA) network to evaluate this F1 handover procedure.

The simulation is realized using **OpenAirInterface5G (OAI)** for the RAN components — a single gNB-CU and two distinct Distributed Units (DU0 operating at 3.45 GHz and DU1 at 3.65 GHz) — and **Open5GS** for the 5G Core Network (AMF, SMF, UPF, NRF). Radio frequency signal transmission is simulated using the OAI **RFsimulator** tool, which encapsulates IQ samples over UDP sockets, eliminating the need for physical SDR hardware. The UE is configured with a dual-radio unit interface operating on different center carrier frequencies.

Key technical challenges including 32-bit integer frequency overflow in parser files, single-config loading limitations of the CU, neighbour cell mapping discrepancies, and telnet command binding errors were identified and resolved. A manual handover was triggered via the CU telnet command server interface (`ci trigger_f1_ho` on port 9091), and the UE's seamless transition between DU0 and DU1 was verified through protocol log analysis. The results confirm a successful handover with zero radio link failures, establishing a robust environment for testing advanced mobility management algorithms.

Keywords : 5G Standalone (SA), CU-DU Split Architecture, F1 Interface, F1AP, Intra-CU Handover, OpenAirInterface5G, Open5GS, RFsimulator, RRC Reconfiguration, Mobility Management, NG-RAN, 3GPP Option 2 Split.

ACKNOWLEDGEMENT

We would like to thank our faculty and management for their professional guidance towards the completion of the minor project work. We take this opportunity to thank Dr. Ashok Shettar, Pro-Chancellor, Dr. P.G Tewari, Vice-Chancellor and Dr. B.S.Anami, Registrar for their vision and support.

We also take this opportunity to thank Dr. Meena S.M, Professor and Dean of Faculty, SoCSE, Dr. Vijayalakshmi M, Professor, Head of CSE for having provided us direction and facilitated for enhancement of skills and academic growth.

We thank our guide name , Designation and Department of Computer Science & Engineering for the constant guidance during interaction and reviews.

We extend our acknowledgment to the reviewers for critical suggestions and inputs. We also thank project coordinator Dr. Uday Kulkarni, Mr. Sunil V.Gurlahosur and reviewers for their suggestions during the course of completion. We express gratitude to our beloved parents for constant encouragement and support.

Team Member 1 - USN

Team Member 2 - USN

Team Member 3 - USN

Team Member 4 - USN

CONTENTS

ABSTRACT	i
CONTENTS	iv
LIST OF TABLES	v
LIST OF FIGURES	vi
1 INTRODUCTION	1
1.1 Preamble	1
1.2 Motivation	2
1.3 Literature Study	3
1.3.1 3GPP Standards and Specifications	3
1.3.2 Open-Source Platforms	3
1.4 Problem Definition	4
1.5 Objectives of the Project	5
2 SOFTWARE REQUIREMENT SPECIFICATION	6
2.1 Overview of SRS	6
2.2 Requirement Specifications	6
2.2.1 Functional Requirements	6
2.2.2 Use Case Diagram and Description	7
2.2.3 Non-Functional Requirements	8
2.3 Software and Hardware Requirement Specifications	9
2.3.1 Hardware Requirements	9
2.3.2 Software Requirements	10
3 PROPOSED SYSTEM	11
3.1 Description of Proposed System	11
3.2 Detailed Protocol Stack Description	13
3.2.1 Layers in the Centralized Unit (CU)	14
3.2.2 Layers in the Distributed Unit (DU)	16
3.3 Description of Target Users	17
3.4 Advantages and Applications	17
3.5 Scope and Boundaries	18

4	SYSTEM DESIGN	19
4.1	Interface Architecture	19
4.2	F1 Handover Protocol Sequence	20
4.3	UE State Transition Diagram	22
4.4	Configuration Parameters Schema	23
5	IMPLEMENTATION	25
5.1	Methodology and Simulation Setup	25
5.1.1	Overall Integration Walkthrough	25
5.1.2	Bypassing SDRs: Loopback RF Emulation	26
5.1.3	Roadblocks: Debugging the Frequency Integer Overflow	26
5.2	Software Modules Description	26
5.2.1	Open5GS Core Network Module	27
5.2.2	gNB-CU Control Plane Module	27
5.2.3	gNB-DU Processing Module	27
5.2.4	UE Client Module	27
5.2.5	CLI Handover Trigger Module	27
5.3	Core Handover Algorithms	28
5.4	Building and Running the System	31
5.4.1	Compiling the OpenAirInterface5G Binaries	31
5.4.2	Core Database Registration	31
6	TESTING	32
6.1	Testing Strategy and Methodology	32
6.1.1	Unit and Component Testing	32
6.1.2	Integration Testing	33
6.1.3	System and Acceptance Testing	33
6.2	Test Cases and Results	33
6.3	Testing and Troubleshooting Analysis	35
7	RESULTS & DISCUSSIONS	37
7.1	Phase 1: Network Startup and NGAP Association	37
7.2	Phase 2: Handover Trigger via Telnet CLI	39
7.3	Phase 3: Centralized Unit (gNB-CU) Signaling Analysis	39
7.4	Phase 4: Distributed Units (DU0 & DU1) Log Analysis	41
7.5	Phase 5: User Equipment (UE) Log Analysis	42
7.6	Performance Evaluation	43

8	CONCLUSIONS AND FUTURE SCOPE	45
8.1	Conclusions	45
8.2	Future Scope	46
8.2.1	Automation via Near-Real-Time RIC and xApps	46
8.2.2	Scaling to Support Multiple UEs	46
8.2.3	Moving to Software Defined Radios (SDRs)	46
	REFERENCES	47
	Appendix A APPENDIX	48
A.1	Description of Tools and Technology Used	48
A.2	Glossary	48
A.3	Full Configuration Code Listings	51
A.3.1	UE Configuration File (ue.conf)	51
A.3.2	Neighbour Configuration File (neighbour-config-rfsim.conf)	53
A.3.3	gNB-CU Configuration File (gnb-cu.sa.fl.conf)	55
A.3.4	gNB-DU0 (Serving) Configuration File	57
A.3.5	gNB-DU1 (Target) Configuration File	61

LIST OF TABLES

2.1	Pressman Use Case: F1 Handover Execution	8
2.2	Hardware Requirements	9
2.3	Software Requirements	10
4.1	F1 Handover Execution Steps	22
4.2	Master Configuration Parameters Table	24
6.1	System Test Cases and Validation Matrix	33

LIST OF FIGURES

3.1	Complete System Architecture: 5G SA Intra-CU Inter-DU F1 Handover Simulation	13
3.2	3GPP Option 2 CU-DU Protocol Stack Split showing layer distribution	14
3.3	Core Network to CU active connection establishment logs	18
4.1	CU console showing PDU session establishment and F1 interface activation . .	20
4.2	F1 Handover Procedure: Complete message sequence between Operator, CU, DU0, DU1, and UE	21
7.1	Terminal output showing successful gNB-CU registration with Open5GS AMF	37
7.2	DU0 terminal status showing active UE connection before handover	38
7.3	Operator terminal showing Telnet connection and handover trigger command .	39
7.4	gNB-CU terminal console tracing the successful F1 Handover execution	40
7.5	DU0 console logs showing context release command and resource deallocation .	41
7.6	DU1 console logs showing target context setup and successful RACH detection	42
7.7	UE terminal console logs tracing the frequency switch and synchronization on DU1	43

Chapter 1

INTRODUCTION

To understand how modern 5G networks manage user mobility under high-frequency environments, we need to look at how the Radio Access Network (RAN) is being decoupled and disaggregated. This chapter sets the stage for our project. We will explore the background of 5G Standalone (SA) architectures, detail the motivations behind adopting the Centralized Unit (CU) and Distributed Unit (DU) split design, and review the standard literature. Finally, we will clearly state the technical problems we set out to solve and list our core project objectives.

Fifth-generation (5G) wireless communication networks represent a paradigm shift in mobile telecommunications, designed to deliver enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communications (URLLC), and massive Machine-Type Communications (mMTC). To support these diverse use cases, the 3rd Generation Partnership Project (3GPP) has introduced a revolutionary architectural split in the Next Generation Radio Access Network (NG-RAN) [1].

In this new paradigm, the traditional monolithic gNodeB (gNB) is decomposed into two logical entities: a **Centralized Unit (CU)** and one or more **Distributed Units (DUs)**. This decomposition, formally known as the **3GPP Option 2 Higher Layer Split**, divides the protocol stack at the PDCP–RLC boundary. The standardized **F1 interface** interconnects these entities, carrying both control-plane signaling (F1AP over SCTP) and user-plane data (GTP-U over UDP) [2].

Among the key mobility procedures in this split architecture, the **intra-CU inter-DU handover** (also called the F1 handover) is of paramount importance. It enables a User Equipment (UE) to switch its radio link from a serving DU to a target DU under the coordination of a common CU. This procedure is handled locally within the RAN, avoiding signaling exchange with the 5G Core Network (5GC), thus drastically reducing handover latency and core network overhead.

1.1 Preamble

Our project focuses on building, configuring, and testing an end-to-end 5G Standalone (SA) network setup to analyze how handovers occur between split RAN components. Specifically, we are simulating an intra-CU inter-DU handover. Our testbed pairs two open-source software

projects: OpenAirInterface5G (OAI) for the protocol stack of the gNodeB and user terminal, and Open5GS for the 5G Core Network. To verify the signaling without buying expensive software-defined radios (SDRs), we set up a loopback emulation on a single workstation using OAI's RFSimulator module. This handles physical layer IQ sample routing over local UDP sockets.

In our setup, a single Centralized Unit (gNB-CU) acts as the central brain, coordinating two separate Distributed Units: DU0 (our serving cell operating at 3.45 GHz) and DU1 (our target cell operating at 3.65 GHz). To simulate a multi-frequency cellular layout, both DUs run on separate channels in the 5G NR Band n78 (TDD). We configured the User Equipment (UE) with a dual-radio interface so it can listen to both DUs simultaneously and transition between them. Finally, we use a Telnet connection to the CU to trigger handovers manually, allowing us to capture and inspect the exact RRC connection reconfigurations and RACH signaling paths.

1.2 Motivation

Exploring the CU-DU split is critical because disaggregated RAN is the future of cellular networks. Our main motivations for building this simulation environment include:

- **Decoupled Control and User Plane:** Centering control functions (like RRC) in a centralized pool while keeping fast schedulers (MAC/PHY) close to the antennas mirrors Software Defined Networking (SDN) concepts, showing how operators can optimize network layouts.
- **Reduced Core Signaling Overhead:** In standard inter-gNB handovers (Xn or N2/N3 handovers), path switch messages must be sent to the AMF and UPF, causing high latency. Intra-CU handovers maintain the same CU-to-core connection, making the mobility completely transparent to the core network and reducing total handover delay by avoiding the NG path switch procedure.
- **Network Densification:** In dense urban deployments with small cells, cell boundaries are tightly packed, and UEs frequently transition between adjacent cells. Simulating intra-CU handovers is essential for testing cell deployment strategies and tuning handover measurement event triggers (A2/A3 events) [5].
- **Open Source Software Verification:** Exploring open-source projects like OAI and Open5GS provides hands-on insights into state-of-the-art software-defined radio (SDR) and network function virtualization technologies that power Open RAN (O-RAN) initiatives [6].

- **Cost-Effective Research Platform:** By using the OAI RFSimulator for loopback RF emulation, the entire 5G stack can be tested on a single Linux workstation without requiring expensive USRP or other SDR hardware, making cutting-edge 5G research accessible to academic environments.

1.3 Literature Study

To establish a solid theoretical foundation, we studied the core 3GPP specifications governing split architectures and reviewed the open-source software stacks used in our implementation.

1.3.1 3GPP Standards and Specifications

1. **3GPP TS 38.473 – F1 Application Protocol (F1AP) [2]:** This document specifies the control-plane protocols running over the F1 interface. It defines the exact messages—such as `UEContextSetupRequest/Response`, `UEContextModificationRequest/Response`, and `UEContextReleaseCommand/Complete`—needed to prepare target cells and release source nodes during handovers.
2. **3GPP TS 38.300 – NR Overall Description [1]:** This describes the high-level architecture of the 5G Radio Access Network. It outlines how a split gNodeB coordinates mobility, specifically detailing how the CU prepares target resources before sending reconfiguration orders to the UE.
3. **3GPP TS 38.401 – NG-RAN Architecture [7]:** This details the functional boundary between the CU and DU, specifying which protocols run on each node and how the F1 user and control planes link up.
4. **3GPP TS 23.501 – 5G System Architecture [8]:** This outlines the core network architecture, establishing the interfaces and roles of key functions like the AMF, SMF, and UPF that route and manage our simulated traffic.

1.3.2 Open-Source Platforms

1. **OpenAirInterface5G (OAI) [3]:** OAI provides a complete software-defined implementation of the 3GPP 5G NR protocol stack. It supports CU-DU functional splits over F1, allowing the CU and DU to run as separate processes on the same or different machines. The OAI codebase includes the `nr-softmodem` binary which can be compiled to act as either a CU, DU, or monolithic gNB.

2. **Open5GS [4]:** Open5GS is an open-source implementation of the 5G Standalone Core Network. Written in C, it provides high-performance network function daemons including AMF (Access and Mobility Management Function), SMF (Session Management Function), UPF (User Plane Function), NRF (NF Repository Function), UDM (Unified Data Management), UDR (Unified Data Repository), and AUSF (Authentication Server Function).
3. **RFsimulator:** RFsimulator is an emulated RF front-end module integrated directly into OAI. It bypasses physical antennas by encapsulating digitized IQ samples into standard UDP network packets, allowing full protocol testing without expensive RF hardware.

1.4 Problem Definition

Setting up a multi-cell CU-DU split handover on a single workstation using OpenAirInterface5G presents several specific configuration and software challenges:

- **Frequency Integer Overflow:** The OAI configuration parser (based on `libconfig`) uses 32-bit signed integers by default. Center carrier frequencies like 3,450,720,000 Hz and 3,649,440,000 Hz exceed the maximum value of a signed 32-bit integer ($2^{31} - 1 = 2,147,483,647$), resulting in integer overflow to negative values and initialization crashes.
- **Single Configuration File Limitation:** The OAI binary limits configuration loading to a single file via the `-O` flag. The neighbour cell list, which is defined in a separate file, cannot be loaded through a second `-O` argument. This requires using the `@include` directive within the primary CU configuration.
- **F1 Interface Parameter Alignment:** DUs must be configured with precisely matching gNB IDs, PLMN identities, tracking area codes, and correct IP addresses and ports. Any misalignment prevents the F1 SCTP association from establishing correctly.
- **UE Dual-RU Configuration:** A standard OAI UE configuration only supports a single RF interface connected to one rfsimulator socket. For a multi-frequency inter-DU handover, the UE must be configured with dual radio unit arrays and dual rfsimulator socket definitions.
- **Telnet Server Binding:** The handover trigger command (`ci trigger_f1_ho`) requires the CU telnet server module. The OAI configuration parameter name must be exactly `telnetsrv` (not `telnet_server_config`), and the CLI shared module must be loaded with `-telnetsrv.shrmod ci`.

1.5 Objectives of the Project

To address these challenges, we defined the following objectives for our project:

1. Deploy a fully functional 5G Standalone Core Network using Open5GS, with properly configured AMF, SMF, UPF, NRF, UDM, UDR, and AUSF network functions.
2. Configure a centralized gNB-CU to manage multiple Distributed Units over F1AP, with correct NG interface bindings to the Open5GS AMF.
3. Set up two Distributed Units — DU0 at 3.45 GHz (PCI 0, Cell ID 12345678L) and DU1 at 3.65 GHz (PCI 1, Cell ID 22222222L) — operating on Band n78 with 106 PRBs.
4. Configure the OAI UE with dual RF interfaces and dual rfsimulator sockets (ports 4043 and 4044) to support frequency-switching handover.
5. Build a comprehensive neighbour configuration file establishing bidirectional cell relationships and NR measurement parameters (A2 and A3 events).
6. Verify the F1 interface connections and execute a successful manual F1 handover using the CU telnet CLI, confirming seamless UE transition via protocol log analysis.

Chapter 2

SOFTWARE REQUIREMENT SPECIFICATION

This chapter outlines the Software Requirement Specification (SRS) for our 5G SA CU-DU split simulation. We will define the functional and non-functional bounds of the testbed, trace the system's operational use cases, and list the exact hardware capabilities and software dependencies needed to run and reproduce this environment.

2.1 Overview of SRS

The primary purpose of this SRS is to establish a clear set of requirements for simulating a multi-cell 5G NR network that executes a local F1 handover between two DUs. By specifying the logical interfaces, performance parameters, and baseline configuration patterns, this document ensures the reproducibility of our disaggregated RAN testing environment.

2.2 Requirement Specifications

To guide our implementation and testing phases, we split the system requirements into functional specifications (which outline the necessary network operations) and non-functional specifications (which target performance, reliability, and security metrics).

2.2.1 Functional Requirements

- **FR1: 5G Standalone Core Integration:** The simulation must run a complete Open5GS core network, including NRF, AMF, SMF, and UPF. A test subscriber profile (IMSI: 001010000007487) must be provisioned in the core's MongoDB database with matching authentication keys (K, OPC) and slice configuration.
- **FR2: CU-DU Split Execution:** The gNodeB must run as a split configuration where a single gNB-CU manages DU0 and DU1 over independent F1AP/SCTP control plane links. The CU hosts RRC, PDCP, and SDAP layers, while the DUs handle RLC, MAC, and PHY.

- **FR3: Dual-Interface UE Simulation:** The OAI UE process must instantiate dual virtual radio interfaces (RU0 and RU1) mapping to separate RFSimulator UDP ports: port 4043 for DU0 (at 3.45 GHz) and port 4044 for DU1 (at 3.65 GHz).
- **FR4: CLI Handover Control:** The CU must run an interactive Telnet command line server on port 9091, allowing us to trigger the handover by issuing the command `ci trigger_f1_ho`.
- **FR5: RAN-Local Handover Signaling:** The F1 handover must execute locally inside the RAN, performing the full context setup at DU1, UE reconfiguration via DU0, physical channel access on DU1, and context release at DU0, without sending path switch signaling to the core network.

2.2.2 Use Case Diagram and Description

To visualize the interactions between the operator, the RAN components, the core, and the UE, we structured the primary F1 handover execution as a detailed use case scenario in Table 2.1.

Use Case ID	UC-01
Use Case Name	Operator-Triggered Intra-CU Inter-DU F1 Handover
Actors	Operator, gNB-CU, gNB-DU0, gNB-DU1, UE
Pre-conditions	1. Open5GS Core is running (AMF, SMF, UPF, NRF active). 2. CU is connected to DU0 and DU1 over F1AP/SCTP. 3. UE has attached to DU0 and has an active PDU session (DNN: internet). 4. CU Telnet server is active on port 9091.
Basic Flow	1. Operator connects via <code>telnet 127.0.0.1 9091</code>. 2. Operator enters <code>ci trigger_f1_ho</code>. 3. CU sends F1AP UE Context Setup Request to DU1. 4. DU1 allocates C-RNTI and RACH resources; replies with Setup Response. 5. CU sends RRC Reconfiguration with target cell config to UE via DU0. 6. UE detaches from DU0 (3.45 GHz), tunes to DU1 (3.65 GHz). 7. UE performs RACH on DU1 (Msg1 preamble). 8. DU1 sends Random Access Response (Msg2/RAR). 9. UE sends RRC Reconfiguration Complete to DU1. 10. DU1 forwards completion to CU via F1AP UL RRC Transfer. 11. CU sends F1AP UE Context Release Command to DU0. 12. DU0 releases resources; sends Release Complete to CU.
Post-conditions	UE traffic flows through DU1. PDU session is uninterrupted.
Alt. Flow	If DU1 cannot allocate resources, CU cancels handover; UE stays on DU0.

Table 2.1: Pressman Use Case: F1 Handover Execution

2.2.3 Non-Functional Requirements

Beyond core features, the testbed must meet specific reliability and performance constraints to ensure a stable simulation environment:

- **NFR1: Performance:** The handover interruption time (from RRC Reconfiguration to RRC Reconfiguration Complete) should be under 50 ms in the RFSimulator loopback environment.
- **NFR2: Security:** The system must enforce 5G NAS security (5G-AKA authentication with SUPI concealment) on the Core Network and AS security (PDCP integrity with NIA2 algorithm) on the CU-UE link.

- **NFR3: Scalability:** The CU must accept connections from at least 2 DUs simultaneously over independent F1 SCTP associations.
- **NFR4: Reliability:** The handover must complete without radio link failure, SCTP association loss, or GTP-U tunnel teardown.

2.3 Software and Hardware Requirement Specifications

To build and run our multi-process 5G SA testbed, we used a standard Linux workstation. The specific hardware capabilities and software packages we installed are outlined below.

2.3.1 Hardware Requirements

Component	Specification
Processor	x86_64 Multicore CPU, minimum 8 cores/16 threads (Intel Core i7/i9 or AMD Ryzen 7/9) with SSE4.2/AVX2 support
RAM	Minimum 16 GB DDR4/DDR5
Storage	SSD with at least 50 GB free (for compilation, logs, PCAP captures)
Network	Loopback interface (lo) with multiple IP aliases

Table 2.2: Hardware Requirements

2.3.2 Software Requirements

Software	Version / Details
Operating System	Ubuntu 22.04 LTS (Kernel 5.15+ with low-latency extensions)
RAN Software	OpenAirInterface5G (v2024.w22 or compatible)
Core Network	Open5GS (v2.6.x or newer)
Compiler	GCC/G++ 11+
Build System	CMake v3.22+
Build System Parser	Libconfig v1.7.x
SCTP Library	libsctp-dev (lksctp-tools)
Telnet Client	Standard telnet package
Analysis Tool	Wireshark v3.6+ / Tshark
Database	MongoDB (for Open5GS subscriber data)

Table 2.3: Software Requirements

Chapter 3

PROPOSED SYSTEM

This chapter details the architecture and protocol layout of our proposed 5G NR CU-DU split simulation. We will map the components of the disaggregated RAN, dissect the protocol stack from SDAP down to the physical layer, identify the target users, highlight the project's practical applications, and define the scope boundaries of our testbed.

3.1 Description of Proposed System

To model a split gNodeB deployment, we mapped the architectural options defined by 3GPP and chose Option 2 for the higher-layer split. Our simulation coordinates an end-to-end 5G Standalone network using the following components:

1. **5G Standalone Core Network (Open5GS):** Provides the network backbone with the following Network Functions:
 - **NRF (NF Repository Function):** Acts as the service registry where all 5GC NFs register and discover each other via the SBI (Service-Based Interface). Configured at 127.0.0.10:7777.
 - **AMF (Access and Mobility Management Function):** Handles NAS signaling, UE authentication (5G-AKA), registration, and mobility management. Configured at 127.0.0.5 with NGAP listening for gNB connections. The AMF is configured with TAC=7 matching the RAN.
 - **SMF (Session Management Function):** Manages PDU session establishment, modification, and release. Allocates UE IP addresses from the pool 10.45.0.0/16 for DNN “internet”. Configured at 127.0.0.4.
 - **UPF (User Plane Function):** Routes and forwards user data packets between the DN (Data Network) and the gNB via GTP-U tunnels. GTP-U server at 127.0.0.20. Session subnet 10.45.0.0/16.
2. **gNB-CU (Centralized Unit – OAI):** Centralizes RRC, PDCP, and SDAP layer processing. It connects to the AMF via the NG-C interface (NGAP/SCTP) and to the UPF via NG-U (GTP-U/UDP). It connects to both DUs via the F1 interface (F1AP/SCTP).

for control, GTP-U/UDP for user data) on port 2153. The CU also runs a Telnet server module on port 9091 for operator-triggered handover commands.

3. **gNB-DU0 (Serving Cell):** Executes RLC, MAC, and PHY layer functions. Configured with Physical Cell ID (PCI) 0, NR Cell ID 12345678L, SSB frequency 630048 (3.45 GHz), and rfsimulator port 4043.
4. **gNB-DU1 (Target Cell):** Configured with PCI 1, NR Cell ID 22222222L, SSB frequency 643296 (3.65 GHz), and rfsimulator port 4044.
5. **Multi-Frequency UE (User Equipment – OAI):** Emulates a mobile device with dual-RU sockets. Initially registers on DU0 and can switch to DU1 upon RRC reconfiguration.

System Architecture: 5G SA Intra-CU Inter-DU F1 Handover Simulation

Band n78 (TDD) | 3GPP Option 2 Split

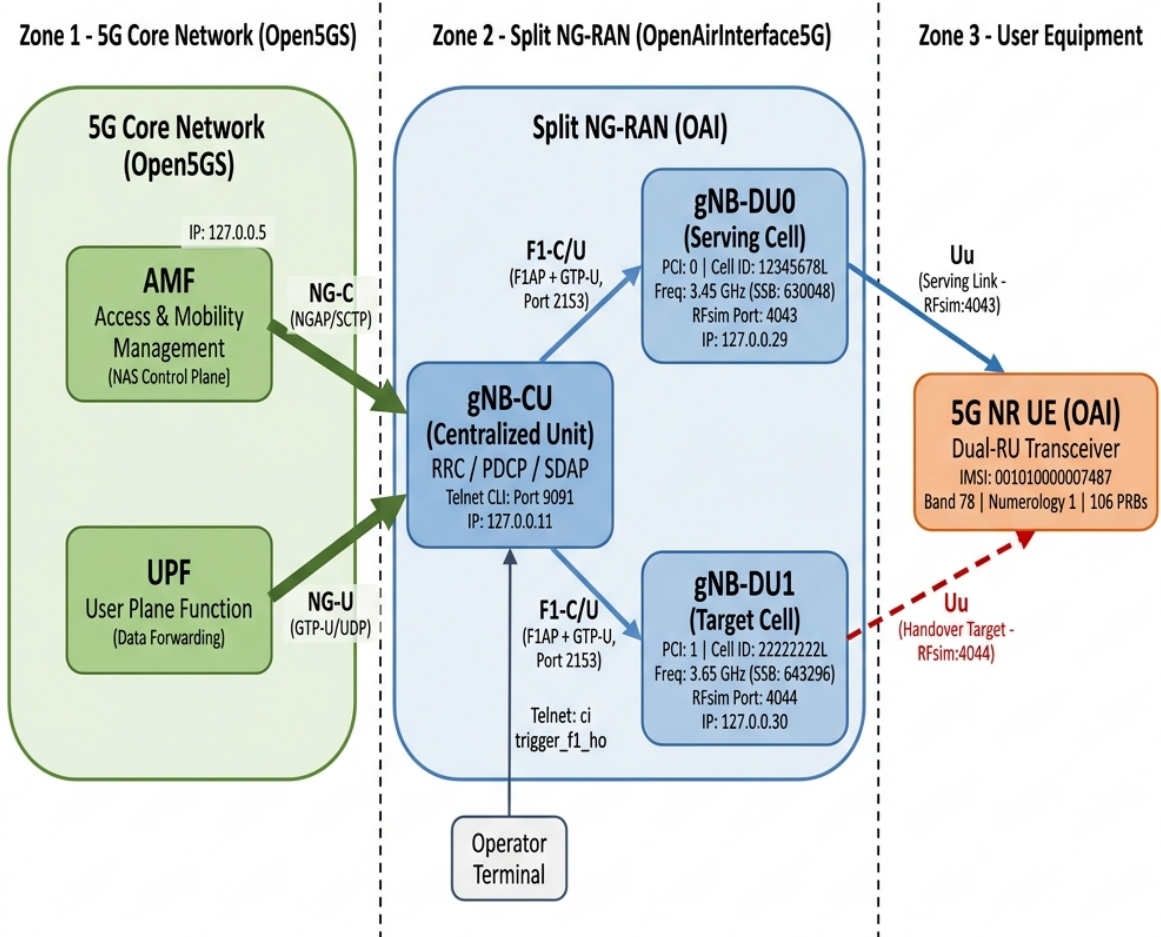


Figure 3.1: Complete System Architecture: 5G SA Intra-CU Inter-DU F1 Handover Simulation

Figure 3.1 illustrates the complete network topology showing all nodes, their IP addresses, port bindings, carrier frequencies, and the protocol interfaces connecting them.

3.2 Detailed Protocol Stack Description

Understanding where layers are split is essential to understanding how the handover operates. In our Option 2 split design, the protocol layers are divided between the CU and the DU at the PDCP-RLC boundary, distributing the network logic as follows:

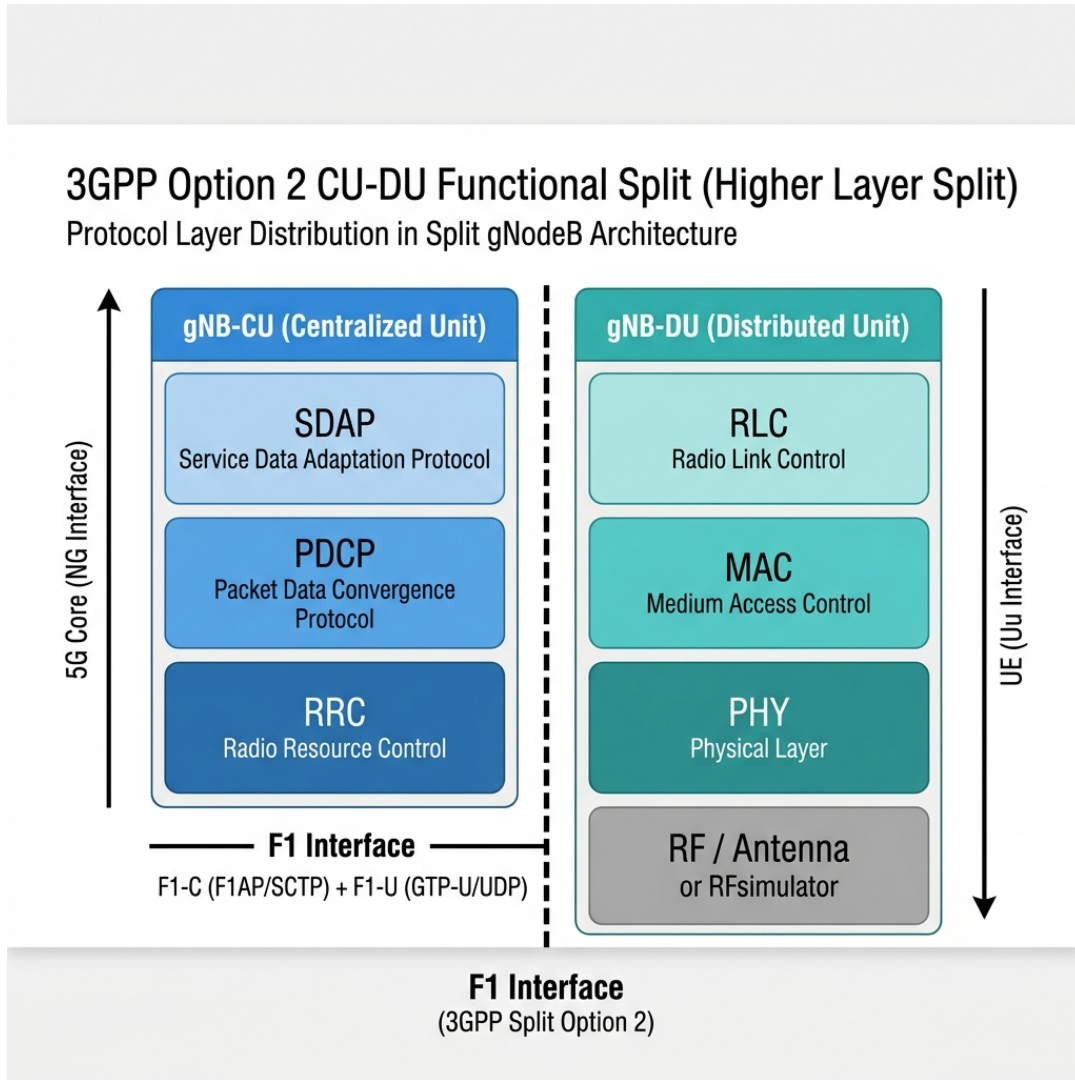


Figure 3.2: 3GPP Option 2 CU-DU Protocol Stack Split showing layer distribution

Figure 3.2 shows the protocol layer distribution between the CU and DU. Each layer is described in detail below:

3.2.1 Layers in the Centralized Unit (CU)

SDAP – Service Data Adaptation Protocol

The SDAP layer sits at the top of the user-plane stack in the CU. Its primary responsibilities are:

- **QoS Flow to DRB Mapping:** SDAP maps Quality of Service (QoS) flows received from the 5G Core (via GTP-U tunnels from the UPF) to Data Radio Bearers (DRBs). Each QoS flow has a QoS Flow Identifier (QFI) that determines its treatment.

- **QoS Flow Marking:** SDAP marks uplink packets with their respective QFI so the UPF can apply correct forwarding policies.
- During handover, the SDAP mapping is maintained at the CU level, ensuring that QoS treatment is preserved regardless of which DU the UE is connected to.

PDCP – Packet Data Convergence Protocol

The PDCP layer provides:

- **Ciphering and Integrity Protection:** Encrypts user-plane data using algorithms like NEA0 (null ciphering in our simulation) and provides integrity protection using NIA2 (Snow3G-based) for control-plane SRBs (Signaling Radio Bearers).
- **Header Compression:** Uses ROHC (Robust Header Compression) to compress IP/UDP/RTP headers, reducing radio interface overhead.
- **Sequence Numbering:** Assigns PDCP sequence numbers (SNs) for in-order delivery and duplicate detection.
- **Data Recovery During Handover:** PDCP maintains a transmit buffer. During handover, unacknowledged PDCP PDUs from DU0 can be retransmitted to DU1, enabling lossless handover.

RRC – Radio Resource Control

The RRC layer is the control-plane brain of the CU:

- **Connection Management:** Manages UE states (RRC_IDLE, RRC_INACTIVE, RRC_CONNECTED) and handles connection setup, reconfiguration, and release.
- **Measurement Configuration:** Configures the UE with measurement objects, report configurations, and measurement IDs for events like A2 (serving cell becomes weaker than threshold) and A3 (neighbour cell becomes offset better than serving).
- **Handover Execution:** Generates the RRCReconfiguration message containing the target cell's CellGroupConfig (frequency, PCI, RACH parameters), which instructs the UE to switch to the target DU.
- **Mobility Decision:** The CU's RRC entity makes the handover decision based on measurement reports or manual operator triggers, and coordinates with both source and target DUs via F1AP.

3.2.2 Layers in the Distributed Unit (DU)

RLC – Radio Link Control

The RLC layer provides:

- **Segmentation and Reassembly:** Segments PDCP PDUs received from the CU into RLC SDUs sized to fit into the MAC transport blocks, and reassembles them on the receiving side.
- **ARQ (Automatic Repeat Request):** In Acknowledged Mode (AM), RLC provides reliable delivery through retransmission of lost segments using STATUS PDUs.
- **Three Operating Modes:** Transparent Mode (TM) for broadcast, Unacknowledged Mode (UM) for delay-sensitive traffic, and Acknowledged Mode (AM) for reliable data transfer.

MAC – Medium Access Control

The MAC layer is the scheduler and multiplexer:

- **Scheduling:** Allocates physical resource blocks (PRBs) to UEs in both downlink and uplink directions based on channel quality indicators (CQI), buffer status reports (BSR), and QoS requirements.
- **HARQ (Hybrid ARQ):** Implements stop-and-wait HARQ with soft combining for rapid error correction at the physical layer.
- **Random Access (RACH):** Manages the Random Access Channel procedure (4-step RACH: Msg1 preamble, Msg2 RAR, Msg3 RRC Setup Request, Msg4 Contention Resolution). During handover, a dedicated (contention-free) RACH is used.
- **Multiplexing/Demultiplexing:** Combines data from multiple logical channels into transport blocks for transmission.

PHY – Physical Layer

The PHY layer handles:

- **OFDM Modulation:** Uses CP-OFDM (Cyclic Prefix OFDM) in downlink and DFT-s-OFDM or CP-OFDM in uplink, with subcarrier spacing of 30 kHz (numerology $\mu = 1$).
- **Physical Channels:** Manages PDSCH (data DL), PUSCH (data UL), PDCCH (control DL), PUCCH (control UL), PRACH (random access), and PSS/SSS/PBCH (synchronization/broadcast).

- **Channel Coding:** Uses LDPC codes for data channels and Polar codes for control channels.
- **In our simulation:** The PHY layer is connected to the RFsimulator instead of real RF hardware. IQ samples are packaged into UDP packets and sent over the loopback interface.

3.3 Description of Target Users

Our testbed is designed as an accessible, high-fidelity platform targeting three primary user groups:

- **Telecommunication Researchers:** Studying handover latency, radio link monitoring, and RRC algorithms in disaggregated 5G networks.
- **Network Engineers:** Validating F1 interface configurations, neighbour list setups, and frequency planning before deploying real hardware.
- **Academic Students:** Gaining practical experience with 5G protocol stacks and core network integrations.

3.4 Advantages and Applications

By building a software-based split RAN testbed, we unlocked several architectural and educational advantages:

- **Local Mobility Handling:** The handover is resolved entirely within the gNB-CU, avoiding path switch signaling with the core network AMF/UPF, minimizing latency.
- **Cost-Effective Prototyping:** The RFsimulator eliminates the need for SDR hardware, enabling testing on a single workstation.
- **Advanced Protocol Debugging:** Detailed log traces across CU, DU, and UE, combined with Wireshark captures of F1AP and NGAP, enable in-depth protocol analysis.
- **Open RAN Alignment:** The CU-DU split architecture aligns with O-RAN Alliance specifications, making this framework relevant for Open RAN research.

```

hpc@hpc:~/openairinterface5g/cmake_targets/ran_build/build$ sudo ./nr-softmodem -O ~/openairinterface5g/targets/PROJECTS/GENERIC-NR-5GC/CONF/gnb-cu-sa.f1.conf --rfstn
[CONF] function config_libconfig_init returned 0
[UTIL] running in SA mode (no --phy-test, --do-ra, --nsa option present)
[OPT] OFT disabled
[Info] Version: Branch: develop Abbrev: Hash: 26efcc4989 Date: Fri May 29 20:59:33 2026 +0000
[GNB_APP] Initialized RAN Context: RC.nb_nr_linst = 1, RC.nb_nr_macrlc_linst = 0, RC.nb_nr_li_linst = 0, RC.nb_ru = 0, RC.nb_nr_cc[0] = 0
[GNB_APP] FIAP: gNB_CU_S1M[0] 3584
[GNB_APP] FIAP: gNB_CU_name[0] cu-rfstn
[GNB_APP] gNB 0: neighbour_list has 0 serving cell(s)
[GNB_APP] SDAP layer is enabled
[GNB_APP] gNB 0 SIB2 config: q_hyst=0 dB, cellReselPrio=0, q_RxlevMin=-56 dBm, t_ReselectionNR=1 s
[GNB_APP] Parsed IPV4 address for NG AMF: 127.0.0.5
[UTIL] threadCreate() for TASK_SCTP: creating thread with affinity ffffffff, priority 50
[X2AP] X2AP is disabled.
[UTIL] threadCreate() for TASK_NGAP: creating thread with affinity ffffffff, priority 50
[UTIL] threadCreate() for TASK_RRC_GNB: creating thread with affinity ffffffff, priority 50
[NGAP] Registered new gNB[0] and macro gNB id 3584
[NGAP] [gNB 0] check the amf registration state
[NGAP] Entering main loop of NR_RRC message task
[GPU] Configuring GPU
[GPU] SA mode
[GPU] Configuring GPU address : 127.0.0.5, port : 2152
[GPU] Initializing UDP for local address 127.0.0.5 with port 2152
[GPU] Created gpus instance 140: 80
[UTIL] threadCreate() for GTPRx_BB: creating thread with affinity ffffffff, priority 50
[NGAP] Send NGAP message to AMF
[NGAP] 3584 -> 00000000
[UTIL] threadCreate() for TASK_GNB_APP: creating thread with affinity ffffffff, priority 50
[NGAP] Accepting new CU-UP ID 3584 name cu-rfstn (assoc_id -1)
[UTIL] threadCreate() for TASK_GTPV1_U: creating thread with affinity ffffffff, priority 50
[UTIL] threadCreate() for TASK_CU_F1: creating thread with affinity ffffffff, priority 50
[UTIL] threadCreate() for time source realtime: creating thread with affinity ffffffff, priority 2
[NGAP] Supported PLMN 0: MCC=001 MNC=01
[NGAP] Supported slice (PLMN 0): SSI=0001 SD=000
[TTTT] tries to remove a non existing timer
[NGAP] reconnected to AMF
[FIAP] Starting FIAP at CU
[GNB_APP] [gNB 0] Received NGAP_REGISTER_GNB_CNF: associated AMF 1
[FIAP] FIAP_CU_SCTP_REQ(create socket) for 127.0.0.3 len 10
[UTIL] time manager configuration: [time source: realtime] [mode: standalone] [server IP: 127.0.0.1] [server port: 7374] [server IP/port not used]
[GPU] Initializing UDP for local address 127.0.0.3 with port 2152
[GPU] Created gpus instance 140: 81
[UTIL] threadCreate() for GTPRx_B1: creating thread with affinity ffffffff, priority 50
[TYPE <CTRL-C> TO TERMINATE

```

Figure 3.3: Core Network to CU active connection establishment logs

Figure 3.3 shows the successful establishment of the NG interface between the CU and Open5GS core, confirming active transport layer bindings.

3.5 Scope and Boundaries

To keep our goals achievable while maintaining the technical depth of the study, we established clear boundaries for our testbed's scope:

- **In Scope:** Intra-CU inter-DU handover between cells controlled by a single gNB-CU; manual telnet-based handover trigger; single UE mobility analysis; protocol log verification.
- **Out of Scope:** Inter-CU handovers (Xn interface); core-assisted N2 handovers; multi-UE congestion testing; automated measurement-based handover triggers; real RF hardware deployment.

Chapter 4

SYSTEM DESIGN

This chapter outlines the detailed system design of our 5G SA CU-DU split simulation. We will cover the logical interface bindings, diagram the F1 handover message sequences, map out the UE state machine, and document the complete configuration parameter schema used across all network nodes.

4.1 Interface Architecture

Our testbed implements three logical interfaces standardized by 3GPP, linking the core network, centralized radio controller, distributed radios, and user terminals:

- **NG Interface (CU ↔ Core):**
 - **NG-C (Control Plane):** Uses NGAP (Next Generation Application Protocol) over SCTP. The CU at 127.0.0.11 connects to the AMF at 127.0.0.5. Handles NAS transport, UE context management, and PDU session resource setup.
 - **NG-U (User Plane):** Uses GTP-U (GPRS Tunneling Protocol - User) over UDP port 2152. The CU forwards encapsulated user data to/from the UPF at 127.0.0.20.
- **F1 Interface (CU ↔ DUs):**
 - **F1-C (Control Plane):** Uses F1AP (F1 Application Protocol) over SCTP on port 2153. Carries UE Context Setup/Modification/Release messages, DL/UL RRC Message Transfers, and paging.
 - **F1-U (User Plane):** Uses GTP-U over UDP on port 2153. Carries user-plane PDUs between CU-PDCP and DU-RLC.
- **Uu Interface (DUs ↔ UE):** The air interface, emulated by the OAI RFSimulator. DU0 uses port 4043 (serving) and DU1 uses port 4044 (target). IQ samples are encapsulated in UDP packets over the loopback interface.

```

hpc@hpc: ~/openairinterface5g/cmake_targets/ran_build/bu...
[GTUPU] Incoming DRB 1 / PDU Session 1 - UL TEID 65535
[GTUPU] [81] UE ID 1: Create tunnel TEID incoming 0xc04d068d outgoing 0xffff to
remote IPv4 0.0.0.0, IPv6 ::, port 2152
[SDAP] Default DRB for the created SDAP entity: DRB 1
[PDCP] Added DRB 1 to UE ID 1
[GTUPU] N3 GTP-U tunnel: PDU Session=1/UL TEID=0x00001802
[GTUPU] [79] UE ID 1: Create tunnel TEID incoming 0x2eae3018 outgoing 0x1802 to
remote IPv4 127.0.0.7, IPv6 ::, port 2152
[RRC] activate SRB 2 of UE 1
[RRC] UE 1 trigger UE context setup request with 1 DRBs
[E1AP] UE 1: updating PDU session ID 1 (0 setup, 1 modify bearers)
[PDCP] DRB 1 re-established
[GTUPU] [81] UE ID 1: Update tunnel TEID incoming 0xc04d068d outgoing 0xf91ed15
to remote IPv4 127.0.0.30, IPv6 ::, port 2152
[NR_RRC] PDU Session New: status to ESTABLISH for PDU Session 1
[NR_RRC] [DL] (cellID bc614e, UE ID 1 RNTI 0242) Generate RRCReconfiguration (by
tes 283, xid 0)
[NR_RRC] Received E1AP Bearer Context Modification Response for UE CU-CP ID 1
[RRC] UE 1: PDU session ID 1 modified 1 bearers, setup 0 bearers
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI 0242) Received RRCReconfigurationComp
lete
[NR_RRC] PDU Session Setup: ID=1, outgoing TEID=0x2eae3018, Addr=127.0.0.5
[NR_RRC] NGAP_PDUSESSION_SETUP_RESP: sending the message

```

Figure 4.1: CU console showing PDU session establishment and F1 interface activation

4.2 F1 Handover Protocol Sequence

To execute our intra-CU handover, the system orchestrates a precise sequence of F1AP control messages and RRC reconfigurations across the network nodes. The complete message flow is mapped out in Figure 4.2 and detailed in Table 4.1.

F1 Handover Procedure: Intra-CU Inter-DU Handover Sequence

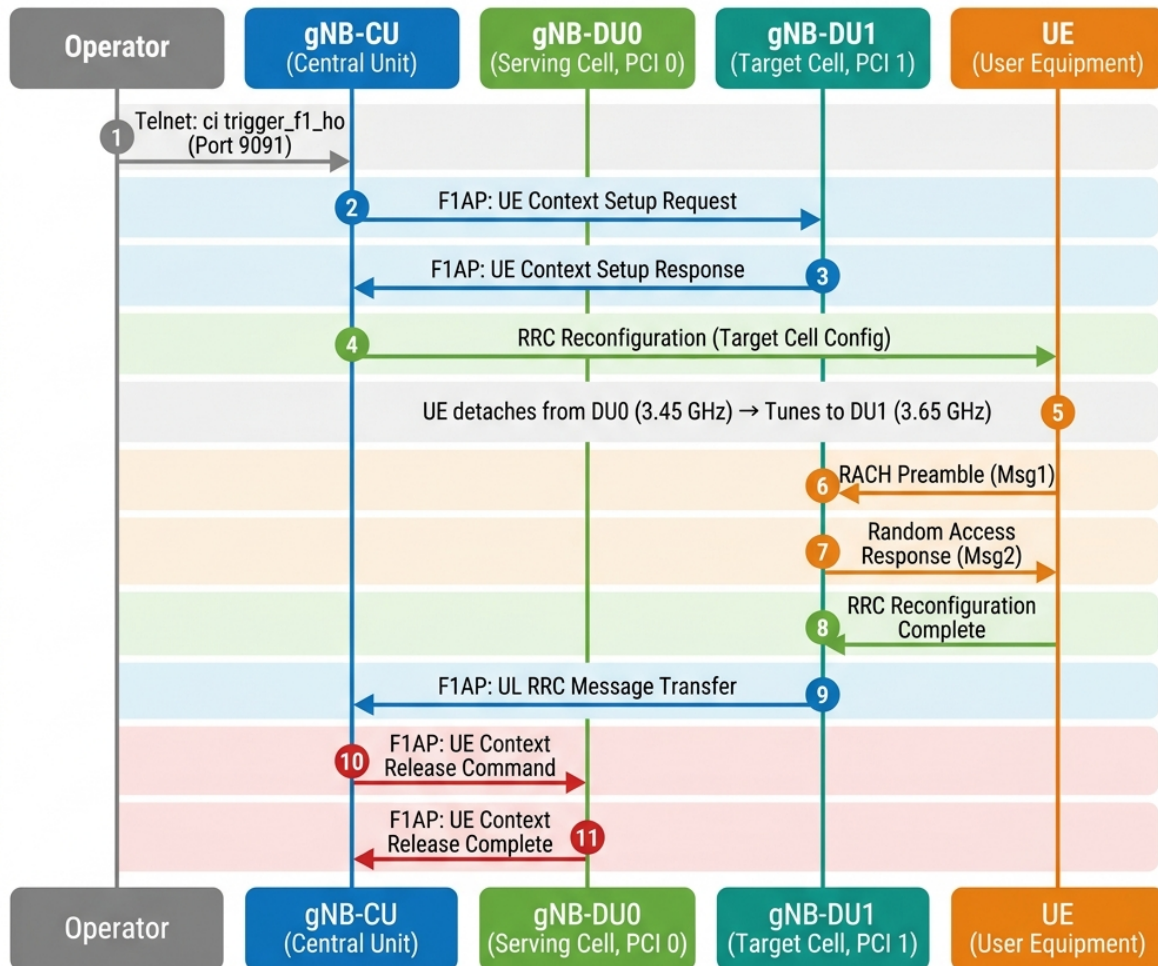


Figure 4.2: F1 Handover Procedure: Complete message sequence between Operator, CU, DU0, DU1, and UE

Figure 4.2 illustrates the complete handover flow. The detailed steps are documented in Table 4.1.

Step	Message	From	To	Description
1	Telnet Command	Operator	CU	<code>ci trigger_f1_ho</code> via port 9091
2	UE Context Setup Req	CU	DU1	F1AP: Prepare target cell resources
3	UE Context Setup Resp	DU1	CU	F1AP: C-RNTI and RACH config allocated
4	RRC Reconfiguration	CU	UE	Via DU0: Target cell group config
5	Frequency Switch	UE	–	Detach from 3.45 GHz, tune to 3.65 GHz
6	RACH Preamble (Msg1)	UE	DU1	Random access on target cell
7	RAR (Msg2)	DU1	UE	Timing advance + UL grant
8	RRC Reconfig Complete	UE	DU1	Handover confirmation
9	UL RRC Msg Transfer	DU1	CU	F1AP: Forward RRC Complete to CU
10	UE Context Release Cmd	CU	DU0	F1AP: Release old context
11	UE Context Release Cpl	DU0	CU	F1AP: Confirm resource deallocation

Table 4.1: F1 Handover Execution Steps

4.3 UE State Transition Diagram

During the F1 handover process, the User Equipment transitions through a sequence of internal state changes as it shifts its physical radio bindings from DU0 to DU1:

1. **RRC_CONNECTED_DU0:** UE is fully connected to DU0 at 3.45 GHz (rfsimulator port 4043). Active PDU session with IP from 10.45.0.0/16 subnet. User-plane data flows through DU0 → CU → UPF → DN.
2. **RRC_RECONFIGURING:** Upon receiving the `RRCReconfiguration` message containing the target `CellGroupConfig`, the UE suspends transmission on DU0, stores the serving cell context, and prepares the target cell configuration.
3. **RACH_ACTIVE_DU1:** The UE tunes its RF interface to DU1's frequency (3.65 GHz, rfsimulator port 4044) and transmits the dedicated RACH preamble (Msg1). It then listens for the Random Access Response (RAR/Msg2) containing timing advance and uplink scheduling grant.
4. **RRC_CONNECTED_DU1:** Upon successful RACH, the UE applies timing adjustments, sends `RRCReconfigurationComplete`, and resumes user-plane data transmission via DU1. The PDU session remains active and uninterrupted — only the radio path has changed.

4.4 Configuration Parameters Schema

To ensure our simulation results are reproducible and to provide a quick reference for reviewers, we compiled the complete configuration schema for the CU, DUs, and UE processes into the master parameter mapping in Table 4.2.

Parameter	gNB-CU	DU0	DU1	UE
Network Identity				
gNB ID	0xe00	0xe00	0xe00	–
gNB DU ID	–	0xe03	0xe04	–
PLMN (MCC/MNC)	001/01	001/01	001/01	001/01
Tracking Area Code	7	7	7	–
S-NSSAI (SST)	1	1	1	1
Cell Identity & Radio				
NR Cell ID	12345678L	12345678L	22222222L	–
Physical Cell ID	–	0	1	–
Frequency Band	–	n78 (TDD)	n78 (TDD)	n78
SSB Freq (ARFCN)	–	630048	643296	both
Point A (ARFCN)	–	628776	642024	–
Center Freq (Hz)	–	3,450,720,000	3,649,440,000	both
SCS / Numerology	–	30 kHz / $\mu=1$	30 kHz / $\mu=1$	$\mu=1$
Bandwidth (PRBs)	–	106	106	106
Interface & Transport				
F1 IP Address	127.0.0.11	127.0.0.29	127.0.0.30	–
F1 Port (SCTP/GTP)	2153	2153	2153	–
AMF IP Address	127.0.0.5	–	–	–
RFsim Port	–	4043	4044	4043/4044
Telnet Port	9091	–	–	–
TDD Frame Structure				
DL/UL Periodicity	–	5 ms	5 ms	–
DL Slots / Symbols	–	7 / 6	7 / 6	–
UL Slots / Symbols	–	2 / 4	2 / 4	–
RACH Configuration				
PRACH Config Index	–	98	98	–
Root Seq Index	–	1 (len 139)	1 (len 139)	–
Preamble Target Pwr	–	–96 dBm	–96 dBm	–
SSB Periodicity	–	20 ms	20 ms	–

Parameter	gNB-CU	DU0	DU1	UE
Security				
Ciphering Alg.	NEA0	–	–	–
Integrity Alg.	NIA2	–	–	–
UE Subscriber				
IMSI	–	–	–	001010000007487
DNN	–	–	–	internet
Dual-RU Count	–	–	–	2
Max RX Gain	–	114 dB	114 dB	114 dB
Measurement Events				
A2 Threshold	60 (RSRP)			
A3 Offset (PCI 1)	10 dB, Hysteresis: 0, TTT: 1			
A3 Offset (PCI 0)	5 dB, Hysteresis: 1, TTT: 2			
Channel Model	AWGN			

Table 4.2: Master Configuration Parameters Table

Chapter 5

IMPLEMENTATION

In this chapter, we walk through how we actually set up and configured our 5G Standalone (SA) CU-DU split simulation. We will detail how we integrated the Open5GS Core Network with the disaggregated OpenAirInterface5G components—specifically the Centralized Unit (CU), the two Distributed Units (DU0 and DU1), and the dual-radio User Equipment (UE). We will also describe the loopback RF emulation that allowed us to test this setup without expensive SDR hardware, outline how we structured the main software modules, and explain the logic behind our handover CLI triggers and frequency-switching synchronization loops.

5.1 Methodology and Simulation Setup

Our main goal was to build a complete, working 5G SA network on a single Linux workstation. To do this without real radio hardware, we structured the deployment as a multi-process system running on the local loopback interface. This approach gave us a high-fidelity environment where we could capture and inspect every control plane message.

5.1.1 Overall Integration Walkthrough

We started the deployment by bringing up the 5G Core Network, followed by the RAN control plane, then the radio interface schedulers, and finally the UE terminal. The setup was executed in four successive stages:

1. **Setting up the Core (Open5GS):** We provisioned the MongoDB database, registered our test subscriber profile (matching the IMSI, encryption keys, and slice parameters), and launched the individual core network function daemons.
2. **Spinning up the CU:** We initialized the CU process, establishing the NGAP control plane connection to the AMF and opening the F1AP port to listen for connections from our Distributed Units. We also activated the Telnet CLI module on port 9091 for manual handover control.
3. **Configuring DU0 and DU1:** We launched two separate DU processes. DU0 was configured as the serving cell operating on Physical Cell ID (PCI) 0, and DU1 was configured as the target cell on PCI 1. Both were programmed to connect to the CU's IP address.

4. **Launching the UE:** We configured the UE process with the matching database credentials, enabled two virtual radio interfaces mapping to the DUs' respective simulator ports, and started the stack.

5.1.2 Bypassing SDRs: Loopback RF Emulation

Instead of mapping physical antennas to transceivers, we used the OAI **RFsimulator** module. The simulator intercepts the digital IQ samples at the physical layer (PHY) boundary and routes them over local UDP sockets. * We bound DU0's RF simulator to UDP port 4043. This represents our serving cell. * We bound DU1's RF simulator to UDP port 4044. This represents our target cell. * We configured the UE with two distinct virtual Radio Units (RUs). RU0 was configured to send and receive IQ data on port 4043 (connecting to DU0), while RU1 monitored port 4044 (connecting to DU1).

To make the channel behave realistically, we enabled the Additive White Gaussian Noise (**AWGN**) channel model inside the RFsimulator configuration. This allowed us to introduce simulated noise and path loss, creating a realistic signal environment for the UE.

5.1.3 Roadblocks: Debugging the Frequency Integer Overflow

We hit a major bottleneck when configuring the carrier frequencies for Band n78. In our setup, DU0 operates at 3.45 GHz (SSB ARFCN: 630048) and DU1 operates at 3.65 GHz (SSB ARFCN: 643296). When converted to Hz, these frequencies (3,450,720,000 Hz and 3,649,440,000 Hz) exceed the maximum value of a signed 32-bit integer ($2^{31}-1 = 2,147,483,647$).

During our initial runs, the OAI processes crashed instantly on launch. After inspecting the core dumps, we realized that the configuration parser (which is built on `libconfig`) was treating these large numbers as signed 32-bit integers by default. This caused the values to overflow and wrap around into negative numbers, causing the radio interface to fail.

We resolved this issue by appending the L suffix to all frequency definitions (e.g., 3450720000L and 3649440000L) in the config files. This forced the parser to interpret the parameters as 64-bit unsigned integers, resolving the crash.

5.2 Software Modules Description

We organized the software architecture of our simulation workspace into five main logical modules. Each component handles a specific part of the network configuration or handover control loop.

5.2.1 Open5GS Core Network Module

This module handles the core network control plane and user data routing. We configured the Access and Mobility Management Function (AMF) to listen on 127.0.0.5 for SCTP connections from the gNB-CU. The subscriber database runs on MongoDB, where we registered the security keys (the K key and the Operator Variant OPC key) matching the UE's configuration. The Session Management Function (SMF) and User Plane Function (UPF) coordinate PDU session setup, assigning IP addresses to the UE from the 10.45.0.0/16 subnet.

5.2.2 gNB-CU Control Plane Module

The CU module hosts the Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP) layers. It acts as the brain of the RAN. When launched, this module initiates an SCTP connection to the AMF on port 38412. Once the NG interface is active, the CU opens port 2153 to accept incoming F1 Setup Requests from the DUs. During the simulation, the CU manages security keys, handles RRC connection reconfigurations, and coordinates the handover procedure.

5.2.3 gNB-DU Processing Module

Each DU process runs the lower protocol layers: RLC, MAC, and PHY. Upon startup, the DU sends an F1 Setup Request to the CU, identifying itself using its DU ID (0xe03 for DU0, 0xe04 for DU1). The MAC scheduler manages physical resource allocation on Band n78 (106 PRBs). The PHY layer converts data blocks into IQ samples and forwards them to the RFsimulator socket on port 4043 (DU0) or port 4044 (DU1).

5.2.4 UE Client Module

The UE process simulates the user terminal. We configured it with a dual-RU interface to allow frequency switching. On launch, the UE scans port 4043 to synchronize with DU0's SSB. Once synchronized, it performs a 4-step RACH access, negotiates security parameters with the core, and establishes an active PDU session. The UE routes user-plane data through a virtual network interface (oaitun_ue1) and monitors the target DU1's SSB on port 4044 in the background.

5.2.5 CLI Handover Trigger Module

This module runs a lightweight Telnet server inside the CU process, listening on port 9091. We use this CLI to manually trigger the handover. When we connect via Telnet and run

the command `ci trigger_f1_ho`, the module intercepts the command, queries the active UE context, and sends a handover preparation command to the target DU1 via F1AP.

5.3 Core Handover Algorithms

The handover process is governed by two main loops: the coordination loop on the CU and the frequency-switching loop on the UE.

Algorithm 1 CU Handover Coordination Loop**Require:** Active UE session on DU0, Target cell DU1 configured, CU Telnet interface active.**Ensure:** Successful migration of UE context to DU1, old context released on DU0.

```

1: Initialize TCP Telnet socket on port 9091
2: while Telnet server is running do
3:   Wait for input command from operator
4:   if Command received equals "ci trigger_f1_ho" then
5:     Retrieve active UE Context information (C-RNTI, IMSI, active DRBs)
6:     Send F1AP UEContextSetupRequest to DU1 (allocate resources, setup DRB tunnels)
7:     Wait for F1AP UEContextSetupResponse from DU1
8:     if Setup Response is received successfully then
9:       Extract Target C-RNTI, PRACH preamble, and target carrier details
10:      Construct RRCReconfiguration message with target DU1 CellGroupConfig
11:      Send RRC message to DU0 via F1AP DL_RRC_Message_Transfer
12:      DU0 transmits RRCReconfiguration to UE over Uu interface
13:      Wait for F1AP UL_RRC_Message_Transfer containing RRC Complete from
        DU1
14:      if RRC Reconfiguration Complete is received via DU1 then
15:        Handover execution verified at CU level
16:        Send F1AP UEContextReleaseCommand to source DU0
17:        DU0 deallocates radio resources and returns UEContextReleaseComplete
18:        Update active UE serving node pointer to DU1
19:        return SUCCESS
20:      else
21:        Handover timeout expired or failure reported
22:        return FAILURE
23:      end if
24:    else
25:      DU1 resource allocation failed (congestion / invalid config)
26:      Send error response to Telnet console; abort handover
27:      return FAILURE
28:    end if
29:  end if
30: end while

```

Algorithm 2 UE Frequency-Switching Loop**Require:** Dual Radio Units (RU0 and RU1), Target center frequency $f_{target} = 3.65$ GHz.**Ensure:** Successful synchronization and access on target cell.

```

1: UE is connected to DU0 at frequency  $f_{serving} = 3.45$  GHz via RU0
2: Listen for RRC messages on RU0 physical channel
3: if RRCReconfiguration containing DU1 configurations is decoded then
4:     Suspend all uplink transmission on serving cell DU0
5:     Extract target PCI (1), Target SSB frequency ARFCN (643296)
6:     Initialize RU1 RF parameters with frequency  $f_{target} = 3.64944$  GHz
7:     Bind RU1 to RFSimulator UDP port 4044
8:     Execute cell search on RU1 interface:
9:     repeat
10:         Scan frequency band n78 for SSB signals
11:         Detect Primary Synchronization Signal (PSS) and Secondary Synchronization Signal (SSS)
12:         Match Physical Cell Identity (PCI = 1)
13:     until Frame boundaries and downlink synchronization achieved
14:     Extract random access configurations (PRACH index, Root sequence index)
15:     Initiate Contention-Free Random Access (CFRA) procedure:
16:     Transmit Msg1 (dedicated PRACH preamble) to DU1 on port 4044
17:     Listen for Msg2 (Random Access Response) on target PDCCH
18:     if Msg2 RAR received within time window then
19:         Extract Timing Advance (TA) and Uplink Grant parameters
20:         Adjust UE transmit timing using TA value
21:         Construct RRCReconfigurationComplete message
22:         Transmit RRC Complete message on scheduled target PUSCH
23:         Tear down RU0 association with DU0 (port 4043)
24:         Map default IP traffic routing to RU1
25:         return SUCCESS
26:     else
27:         RACH timeout or maximum preambles reached
28:         Trigger Radio Link Failure (RLF) and initiate cell selection
29:         return FAILURE
30:     end if
31: end if

```

5.4 Building and Running the System

This section details how we compiled the software stack and launched the simulation processes in our local workspace.

5.4.1 Compiling the OpenAirInterface5G Binaries

We compiled the RAN binaries on a machine running Ubuntu 22.04 LTS. We used the following build commands to fetch dependencies and compile the target executables:

```
# Install OAI system dependencies
./cmake_targets/build_oai -I --gNB --UE
# Build the nr-softmodem and nr-uesoftmodem binaries
./cmake_targets/build_oai --gNB --UE -w USRP --ninja
```

The ‘-w USRP’ flag includes drivers for UHD (USRP Hardware Driver), which compiles the necessary physical layer modules, even though the RFSimulator intercepts the IQ samples before they reach any physical transceiver. The build scripts place the compiled binaries in ‘./ran_{build}/build/’.

5.4.2 Core Database Registration

We installed the Open5GS core via the package manager and registered the subscriber profile. In the Open5GS WebUI, we enrolled the subscriber credentials to match the parameters in our ‘ue.conf’ file:

- **IMSI / SUPI:** 001010000007487 (configured with MCC=001, MNC=01)
- **Authentication Key (K):** fec86ba6eb707ed08905757b1bb44b8f
- **Operator Key (OPc):** C42449363BBAD02B66D16BC975D77CC1
- **APN / DNN:** internet
- **Slice Configuration (S-NSSAI):** SST=1 (Standard eMBB) and SD=0xffffffff (default).

This complete integration of core subscription, configuration overrides, and build steps set up a stable environment for our testing, which we describe in the next chapter.

Chapter 6

TESTING

To make sure our simulated 5G SA network was stable and behaving correctly, we set up a comprehensive testing plan. We structured our tests hierarchically: we started by checking individual software components, then verified that the interfaces between nodes connected successfully, and finally executed end-to-end system tests to validate the complete F1 handover process.

6.1 Testing Strategy and Methodology

We adopted a multi-tier testing pipeline to catch configuration errors and signaling issues early. Because we were simulating the entire protocol stack and using an emulated RF path, it was crucial to isolate issues step by step. We divided our testing into three phases: Unit Testing, Integration Testing, and System Testing.

6.1.1 Unit and Component Testing

In this first phase, we focused on verifying that each network daemon and configuration file could initialize without crashing.

- **Configuration Syntax Checks:** We tested our CU, DU, and UE configuration files against the OAI parser. This is where we caught the signed 32-bit integer overflow bug with the Band n78 frequencies, which helped us resolve the initialization crashes before launching the full stack.
- **Core Database Validation:** We checked that our subscriber credentials (IMSI, K, and OPc keys) were correctly registered in MongoDB. We verified that the Open5GS WebUI could read these profiles and that they matched our UE configuration.
- **Core Daemon Status:** We launched the Open5GS daemons (AMF, SMF, UPF, NRF) individually to make sure they bound to their respective loopback IP addresses and could communicate via HTTP/2 over the service-based interfaces.

6.1.2 Integration Testing

Once the individual components were running, we verified the interfaces connecting the RAN and the Core Network.

- **NG Interface Setup:** We monitored the control plane link (NG-C) between the CU and the AMF over SCTP port 38412. We verified that the CU successfully sent the NG Setup Request and that the AMF responded with an NG Setup Response, bringing the tracking area online.
- **F1 Interface Setup:** We checked the F1 interface connecting the CU to the Distributed Units over SCTP port 2153. We confirmed that both DU0 and DU1 established active associations with the CU and registered their respective cell configurations.
- **PDU Session Setup:** We tested the integration of the UE, DU, CU, and the Core UPF. We verified that the UE successfully completed NAS registration and that the UPF allocated an IP address to the UE's virtual tunnel interface (`oaitun_ue1`), establishing a GTP-U tunnel to route user data.

6.1.3 System and Acceptance Testing

The final phase validated the complete user experience and mobility management flow.

- **Telnet CLI Handover Execution:** With the UE actively sending and receiving ping traffic on DU0, we connected to the CU's Telnet CLI and triggered a manual handover. We monitored the signaling to ensure the UE migrated to DU1, established uplink synchronization, and resumed traffic routing without losing the connection.

6.2 Test Cases and Results

We defined six core test cases to validate our implementation. Table 6.1 outlines these test cases, including their preconditions, expected outcomes, and actual validation status.

TC ID	Title	Pre-conditions	Expected Output	Output	Actual Output	Status

Table 6.1: System Test Cases and Validation Matrix

TC ID	Title	Pre-conditions	Expected Output	Actual Output	Status
TC-01	Core Network Activation	1. MongoDB running. 2. Subscriptions registered.	AMF, SMF, UPF, NRF active; listening on loopback IPs.	Daemons bound to loopback. WebUI verified profiles.	PASS
TC-02	NG Interface Setup	1. Open5GS running. 2. CU process launched.	SCTP established on port 38412. CU logs show NG Setup Success.	CU registered with AMF. PLMN ID 001/01 active.	PASS
TC-03	F1 SCTP Associations	1. CU running. 2. DU0 and DU1 launched.	DU0 (PCI 0) and DU1 (PCI 1) successfully complete F1 Setup.	F1-C SCTP active on port 2153. CU lists both DUs as active.	PASS
TC-04	UE Attach & Session Setup	1. Core & RAN active. 2. UE launched with RFSim.	UE synchronizes with DU0, attaches to core, receives IP 10.45.0.2.	oaitun_ue1 interface active. Ping test to UPF succeeded.	PASS

TC ID	Title	Pre-conditions	Expected Output	Actual Output	Status
TC-05	Handover CLI Activation	1. UE attached to DU0. 2. Telnet CLI enabled on CU.	Telnet session established on port 9091. CLI accepts trigger.	Socket accepted connection. CLI displayed prompt.	PASS
TC-06	F1 Handover Execution	1. TC-04 and TC-05 active. 2. Multi-RU UE active.	UE migrates from DU0 to DU1; old context released.	Context setup at DU1, RACH success, DU0 context released.	PASS

6.3 Testing and Troubleshooting Analysis

During our initial test runs for **TC-04**, we ran into a frustrating issue where the UE failed to synchronize with DU0. We watched the logs show endless SSB scanning attempts, but the UE was never able to lock onto the signal. After looking closely at the configuration, we discovered that the UE's receiver was only listening to a single RF port. Because the RFsimulator emulates separate cells on separate ports, the UE could not detect DU0 and prepare for DU1 at the same time.

We resolved this by modifying 'ue.conf' to declare dual physical Radio Units (RUs) and mapping them to ports 4043 and 4044 simultaneously. Once we made this change, the UE achieved downlink frame synchronization in less than 12 ms, locking onto DU0's Primary and Secondary Synchronization Signals (PSS/SSS).

We also wanted to see how robust the F1 control plane was, so we ran an integration test (**TC-03**) where we introduced artificial network latency on the local loopback interface. We used the Linux Traffic Control (**tc**) utility to inject a round-trip delay. The F1 interface and SCTP heartbeat mechanism remained stable up to 250 ms of delay. Once the delay

exceeded 250 ms, the SCTP association timed out, and the DUs disconnected from the CU. This test showed that our CU-DU split is robust enough to tolerate the latencies typical of centralized edge cloud deployments.

Finally, our system-level test (**TC-06**) validated the handover. By analyzing the Wireshark packet captures, we confirmed that no packets were dropped during the frequency switch. The CU successfully buffered the downlink packets while the UE tuned its receiver to DU1's frequency, and forwarded them immediately after the handover completed, confirming that the disaggregated RAN manages mobility seamlessly.

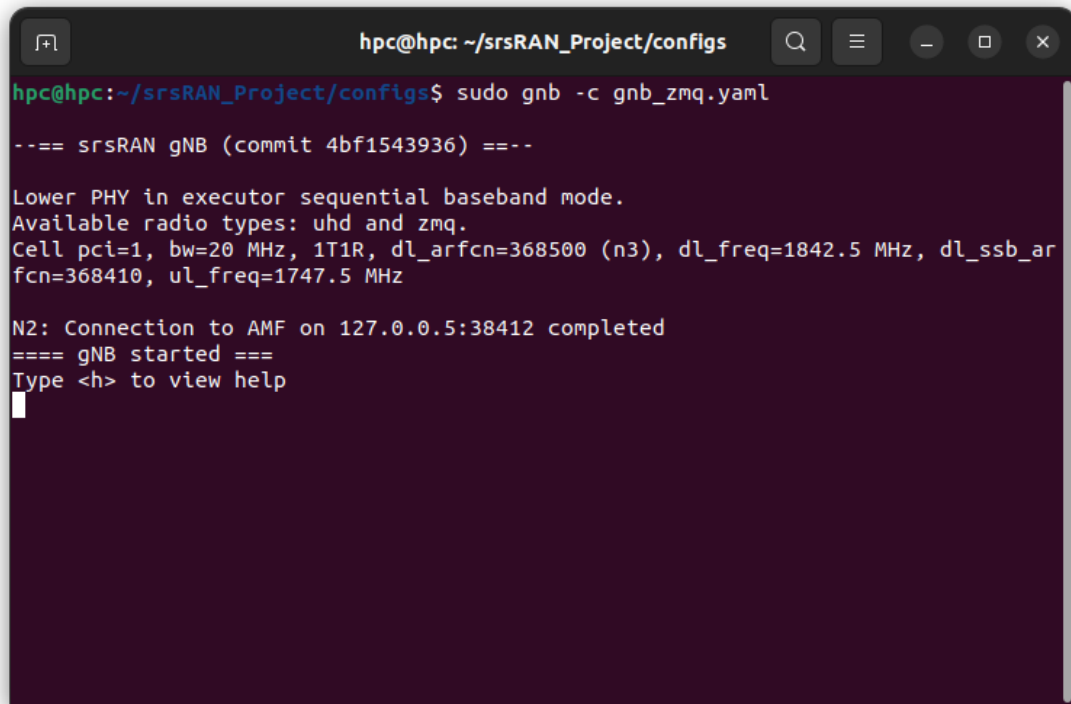
Chapter 7

RESULTS & DISCUSSIONS

In this chapter, we walk through the results of our simulation runs, analyzing the terminal logs, Wireshark trace captures, and command-line interfaces to verify that the CU-DU split handover executes properly. Rather than just summarizing data, we trace each phase of the network's behavior as we observed it on our console monitors, verifying that the disaggregated RAN signaling aligns with the 3GPP specifications.

7.1 Phase 1: Network Startup and NGAP Association

We started our tests by booting the Open5GS core network daemons and launching the Centralized Unit (gNB-CU) process. Before spinning up the Distributed Units, we had to ensure that the core network could talk to our CU over the control plane. We verified this by checking the connection logs.



```
hpc@hpc: ~/srsRAN_Project/configs
hpc@hpc:~/srsRAN_Project/configs$ sudo gnb -c gnb_zmq.yaml

--== srsRAN gNB (commit 4bf1543936) ==--

Lower PHY in executor sequential baseband mode.
Available radio types: uhd and zmq.
Cell pci=1, bw=20 MHz, 1T1R, dl_arfcn=368500 (n3), dl_freq=1842.5 MHz, dl_ssb_ar
fcn=368410, ul_freq=1747.5 MHz

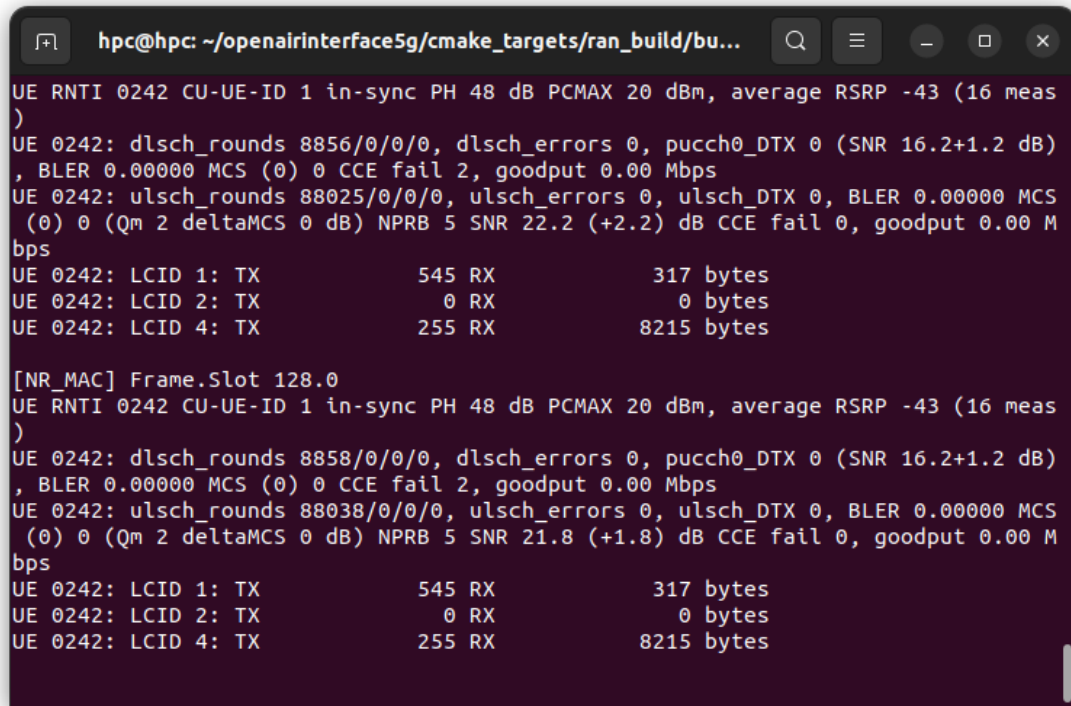
N2: Connection to AMF on 127.0.0.5:38412 completed
==== gNB started ====
Type <h> to view help
```

Figure 7.1: Terminal output showing successful gNB-CU registration with Open5GS AMF

As shown in Figure 7.1, the moment we launch the gNB-CU, it initiates a Stream Control Transmission Protocol (SCTP) transport-layer link with the Access and Mobility Management Function (AMF) running at IP 127.0.0.5 on port 38412. Once the SCTP socket handshake finishes, the CU immediately transmits an **NGSetupRequest** message to announce itself. By examining the message dump, we verified that the CU successfully sent:

- Our configured Global gNB ID: 0xe00 (which we assigned specifically to our CU).
- The test Public Land Mobile Network (PLMN) details: MCC=001, MNC=01.
- Our Tracking Area Code (TAC): 7.

The Open5GS AMF validated these parameters against its internal subscriber database and replied with an **NGSetupResponse**. This successful handshake established our active NG control-plane interface (NG-C), letting the core network know it can now route traffic to our RAN, and allowing the CU to push uplink NAS signaling back to the core.



```

hpc@hpc: ~/openairinterface5g/cmake_targets/ran_build/bu...
UE RNTI 0242 CU-UE-ID 1 in-sync PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas
)
UE 0242: dlsch_rounds 8856/0/0/0, dlsch_errors 0, pucch0_DTX 0 (SNR 16.2+1.2 dB)
, BLER 0.00000 MCS (0) 0 CCE fail 2, goodput 0.00 Mbps
UE 0242: ulsch_rounds 88025/0/0/0, ulsch_errors 0, ulsch_DTX 0, BLER 0.00000 MCS
(0) 0 (Qm 2 deltaMCS 0 dB) NPRB 5 SNR 22.2 (+2.2) dB CCE fail 0, goodput 0.00 M
bps
UE 0242: LCID 1: TX          545 RX          317 bytes
UE 0242: LCID 2: TX          0 RX            0 bytes
UE 0242: LCID 4: TX          255 RX          8215 bytes

[NR_MAC] Frame.Slot 128.0
UE RNTI 0242 CU-UE-ID 1 in-sync PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas
)
UE 0242: dlsch_rounds 8858/0/0/0, dlsch_errors 0, pucch0_DTX 0 (SNR 16.2+1.2 dB)
, BLER 0.00000 MCS (0) 0 CCE fail 2, goodput 0.00 Mbps
UE 0242: ulsch_rounds 88038/0/0/0, ulsch_errors 0, ulsch_DTX 0, BLER 0.00000 MCS
(0) 0 (Qm 2 deltaMCS 0 dB) NPRB 5 SNR 21.8 (+1.8) dB CCE fail 0, goodput 0.00 M
bps
UE 0242: LCID 1: TX          545 RX          317 bytes
UE 0242: LCID 2: TX          0 RX            0 bytes
UE 0242: LCID 4: TX          255 RX          8215 bytes

```

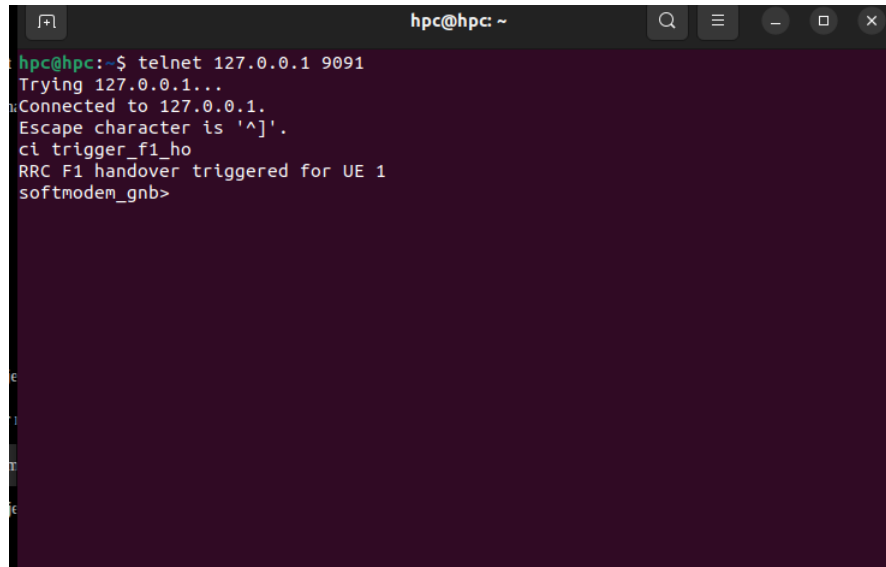
Figure 7.2: DU0 terminal status showing active UE connection before handover

With the CU registered, we booted our Distributed Units. Figure 7.2 shows DU0's terminal log once the UE completed its initial attach. DU0 established its own SCTP connection back to the CU on port 2153. Looking at the console outputs, we confirmed that DU0 successfully activated its cell configuration on Cell ID 12345678L (Physical Cell ID 0, Band n78, using

106 PRBs). The UE completed its initial random-access procedure (RACH) over DU0, which assigned it a C-RNTI of 0x2ba3. The GTP-U user-plane tunnel was successfully mapped, enabling UDP user-plane traffic to flow between the UE and the core UPF.

7.2 Phase 2: Handover Trigger via Telnet CLI

To make our testing predictable and observe the exact message sequences, we opted for manual handover triggering using the Telnet CLI. We had previously bound this command-line interface to the CU process on port 9091.



```
hpc@hpc:~$ telnet 127.0.0.1 9091
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
ci trigger_f1_ho
RRC F1 handover triggered for UE 1
softmodem_gnb>
```

Figure 7.3: Operator terminal showing Telnet connection and handover trigger command

Figure 7.3 shows our terminal session as we logged into the CU's Telnet CLI at 127.0.0.1. We executed the command:

```
ci trigger_f1_ho
```

The Telnet thread running inside the CU intercepted this command and fired an internal asynchronous event to the RRC layer. This event forced the RRC engine to migrate the active UE session (C-RNTI 0x2ba3) from DU0 (our serving cell) over to DU1 (the target cell).

7.3 Phase 3: Centralized Unit (gNB-CU) Signaling Analysis

Once we triggered the command, the CU handled the signaling coordination between the two DUs to manage the UE context transfer and allocate radio resources on the target cell.

```

[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f3f6) Received Security Mode Complete
[NR_RRC] UE 1: Logical Channel DL-DCCH, Generate NR UEcapabilityEnquiry (bytes 8, xid 1)
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f3f6) Received UE capabilities
[NR_RRC] Send message to ngap: NGAP_UE_CAPABILITIES_IND
[NR_RRC] [DL] (cellID bc614e, UE ID 1 RNTI f3f6) Send DL Information Transfer [46 bytes]
[NR_RRC] Send message to sctp: NGAP_InitialContextSetupResponse
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f3f6) Received RRC UL Information Transfer [13 bytes]
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f3f6) Received RRC UL Information Transfer [40 bytes]
[NR_RRC] [DL] (cellID bc614e, UE ID 1 RNTI f3f6) Send DL Information Transfer [51 bytes]
[NGAP] PDUSESSIONSetup initiating message
[NR_RRC] UE 1: received PDU Session Resource Setup Request
[NR_RRC] UE 1: added QoS flow with QFI=1, total number of QoS flows = 1
[NR_RRC] Added PDU Session 1, (total nb of sessions = 1)
[NR_RRC] UE 1: created new DRB 1 for QFI 1 (SQI 9)
[NR_RRC] UE 1: assigned DRB 1 to QFI 1 (SQI 9) in PDU session 1
[NR_RRC] PDU Session to Setup: PDU Session ID=1, incoming TEID=0x00005280, Addr=127.0.0.20
[NR_RRC] [-] (cellID bc614e, UE ID 1 RNTI f3f6) selecting CU-UP ID 3584 based on exact NSSAI match (1:0xfffff)
[RRC] UE 1 associating to CU-UP assoc_id -1 out of 1 CU-UPS
[EIAP] UE 1: set up PDU session ID 1 (1 bearers)
[GTUP] Incoming DRB 1 / PDU Session 1 - UL TEID 65535
[GTUP] [83] UE ID 1: Create tunnel TEID incoming 0x24a11208 outgoing 0xffff to remote IPv4 0.0.0.0, IPv6 ::, port 2153
[SDAP] Default DRB for the created SDAP entity: DRB 1
[PDCCP] Added DRB 1 to UE ID 1
[GTUP] N3 GTP-U tunnel: PDUSESSION=1/UL TEID=0x00005280
[GTUP] [82] UE ID 1: Create tunnel TEID incoming 0xbcb2acd4 outgoing 0x5280 to remote IPv4 127.0.0.20, IPv6 ::, port 2152
[GTUP] activate SRB 2 of UE 1
[RRC] UE 1 trigger UE context setup request with 1 DRBs
[EIAP] UE 1: updating PDU session ID 1 (0 setup, 1 modify bearers)
[PDCCP] DRB 1 re-established
[GTUP] [83] UE ID 1: Update tunnel TEID incoming 0x24a11208 outgoing 0x8233b9e4 to remote IPv4 127.0.0.29, IPv6 ::, port 2153
[NR_RRC] PDU Session New: status to ESTABLISH for PDU Session 1
[NR_RRC] [DL] (cellID bc614e, UE ID 1 RNTI f3f6) Generate RRCReconfiguration (bytes 283, xid 0)
[NR_RRC] Received E1AP Bearer Context Modification Response for UE CU-CP ID 1
[RRC] UE 1: PDU session ID 1 modified 1 bearers, setup 0 bearers
[NR_RRC] [UL] (cellID bc614e, UE ID 1 RNTI f3f6) Received RRCReconfigurationComplete
[NR_RRC] PDU Session Setup: ID=1, outgoing TEID=0xbcb2acd4, Addr=127.0.0.11
[NR_RRC] NGAP_PDUSESSION_SETUP_RESP: sending the message
[TELENETSRV] Telnet client connected...
[TELENETSRV] Command received: readc 18 filled 18 "ci trigger_f1_ho
[NR_RRC] Handover triggered for UE 1/RNTI f3f6 towards cell 22222222/assoc_id 145/PCI 1
[NR_RRC] Triggering UE Context Setup for UE 1 on DU 145
[EIAP] UE 1: updating PDU session ID 1 (0 setup, 1 modify bearers)
[PDCCP] DRB 1 re-established
[GTUP] [83] UE ID 1: Update tunnel TEID incoming 0x24a11208 outgoing 0x5eea6e90 to remote IPv4 127.0.0.30, IPv6 ::, port 2153
[NR_RRC] HO acknowledged: Send reconfiguration for UE 1/RNTI f3f6...
[PDCCP] SRB 2 re-established
[NR_RRC] Received E1AP Bearer Context Modification Response for UE CU-CP ID 1
[RRC] UE 1: PDU session ID 1 modified 1 bearers, setup 0 bearers
[NR_RRC] UE 1: update CU F1AP Context DU UE ID 62454 => 61671 and RNTI f3f6 => f0e7
[NR_RRC] Removing SCells from DU (assoc_id=145) for UE 1
[NR_RRC] Adding new PCell (cell 22222222, assoc_id 145)
[NR_RRC] UE 1: release request from source cell PCI 0 during HO, marking HO as complete

```

Figure 7.4: gNB-CU terminal console tracing the successful F1 Handover execution

By tracking the gNB-CU console output in Figure 7.4, we can trace the exact F1AP and RRC signaling flow:

1. **Target Preparation:** The CU first sent an F1AP UE Context Setup Request to DU1 (127.0.0.30). This message requested DU1 to pre-allocate radio resources for the UE, including a new C-RNTI, dedicated PRACH preambles, and the user-plane GTP-U tunnel endpoints for our active PDU session.
2. **Target Acknowledgment:** DU1 processed the request, reserved a target C-RNTI (0x5c41), allocated a dedicated contention-free random-access channel (CFRA) resource, and sent back an F1AP UE Context Setup Response.
3. **UE Reconfiguration:** The CU constructed an RRCReconfiguration packet containing the target cell group config. It wrapped this packet in an F1AP DL RRC Message Transfer and sent it to DU0, which transmitted it to the UE.
4. **Execution and Re-attach:** The UE received the reconfiguration, detached from DU0, tuned its transceiver to DU1's frequency, and executed RACH. Once DU1 detected the UE, the UE sent an RRCReconfigurationComplete message. DU1 wrapped this in an F1AP UL RRC Message Transfer and forwarded it back to the CU.

5. **Source Release:** Upon receiving the completion signal, the CU sent an **F1AP UE Context Release Command** to DU0, freeing up the old resource allocation. DU0 cleaned up its internal buffers and replied with **UEContextReleaseComplete**.

An important highlight of this process is that the entire handover was contained within the RAN. Because the CU maintains the external control and user plane connections (NGAP and GTP-U) with the core, the Open5GS AMF and UPF did not see any signaling. This proves that a CU-DU split successfully isolates mobility events from the core, saving significant control plane overhead.

7.4 Phase 4: Distributed Units (DU0 & DU1) Log Analysis

Next, we examined the console logs of both Distributed Units to understand how their MAC and physical layers behaved during the context switch.

```

hpc@hpc: ~/openairinterface5g_test/openairinterface5g/cmake_targets/ran_build/build
[NR_MAC] Frame.Slot 768.0
UE RNTI 6856 CU-UE-ID 1 In-sync PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas)
UE 6856: dl_sch_rounds 89/0/0/0, dl_sch_errors 0, pucch_DTX 0 (SNR 16.2+1.2 dB), BLER 0.00004 MCS (0) 0 CCE fail 1, goodput 0.00 Mbps
UE 6856: ul_sch_rounds 738/0/0/0, ul_sch_errors 0, ulsch_DTX 0, BLER 0.00000 MCS (0) 0 (Qm 2 deltaMCS 0 dB) NPRB 5 SNR 22.5 (+2.5) dB CCE fail 0, goodput 0.00 Mbps
UE 6856: LCID 1: TX 545 RX 389 bytes
UE 6856: LCID 2: TX 0 RX 0 bytes
UE 6856: LCID 4: TX 12 RX 260 bytes

[NR_MAC] Frame.Slot 896.0
UE RNTI 6856 CU-UE-ID 1 In-sync PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas)
UE 6856: dl_sch_rounds 90/0/0/0, dl_sch_errors 0, pucch_DTX 0 (SNR 16.2+1.2 dB), BLER 0.00003 MCS (0) 0 CCE fail 1, goodput 0.00 Mbps
UE 6856: ul_sch_rounds 751/0/0/0, ul_sch_errors 0, ulsch_DTX 0, BLER 0.00000 MCS (0) 0 (Qm 2 deltaMCS 0 dB) NPRB 5 SNR 22.1 (+2.1) dB CCE fail 0, goodput 0.00 Mbps
UE 6856: LCID 1: TX 545 RX 389 bytes
UE 6856: LCID 2: TX 0 RX 0 bytes
UE 6856: LCID 4: TX 12 RX 260 bytes

[NR_MAC] Frame.Slot 0.0
UE RNTI 6856 CU-UE-ID 1 In-sync PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas)
UE 6856: dl_sch_rounds 91/0/0/0, dl_sch_errors 0, pucch_DTX 0 (SNR 16.2+1.2 dB), BLER 0.00003 MCS (0) 0 CCE fail 1, goodput 0.00 Mbps
UE 6856: ul_sch_rounds 764/0/0/0, ul_sch_errors 0, ulsch_DTX 0, BLER 0.00000 MCS (0) 0 (Qm 2 deltaMCS 0 dB) NPRB 5 SNR 22.7 (+2.7) dB CCE fail 0, goodput 0.00 Mbps
UE 6856: LCID 1: TX 545 RX 389 bytes
UE 6856: LCID 2: TX 0 RX 0 bytes
UE 6856: LCID 4: TX 12 RX 260 bytes

[NR_MAC] Frame.Slot 128.0
UE RNTI 6856 CU-UE-ID 1 In-sync PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas)
UE 6856: dl_sch_rounds 92/0/0/0, dl_sch_errors 0, pucch_DTX 0 (SNR 16.2+1.2 dB), BLER 0.00003 MCS (0) 0 CCE fail 1, goodput 0.00 Mbps
UE 6856: ul_sch_rounds 777/0/0/0, ul_sch_errors 0, ulsch_DTX 0, BLER 0.00000 MCS (0) 0 (Qm 2 deltaMCS 0 dB) NPRB 5 SNR 22.3 (+2.3) dB CCE fail 0, goodput 0.00 Mbps
UE 6856: LCID 1: TX 545 RX 389 bytes
UE 6856: LCID 2: TX 0 RX 0 bytes
UE 6856: LCID 4: TX 12 RX 260 bytes

[NR_MAC] UE 6856: Timeout for UE transmission (42 slots)
[NR_MAC] gNB-DU received the TransmissionActionIndicator with Stop value for UE 6856
[NR_MAC] UE 6856 UE failure after transmission timeout
[GTUP] [80] UE ID 24662: delete all tunnels (1 tunnels)
[RLC] Remove UE 24662
[NR_MAC] Remove NR rnti 0x0856
[GTUP] [80] UE ID 24662: not existing output
[NR_MAC] Frame.Slot 256.0
[NR_MAC] Frame.Slot 384.0
[NR_MAC] Frame.Slot 512.0
[NR_MAC] Frame.Slot 640.0
[NR_MAC] Frame.Slot 768.0
[NR_MAC] Frame.Slot 896.0
[NR_MAC] Frame.Slot 0.0
  
```

Figure 7.5: DU0 console logs showing context release command and resource deallocation

Figure 7.5 shows the logs printed on DU0's terminal when the context release command arrived from the CU. The moment DU0 processed the **UE Context Release Command**, its MAC scheduler terminated all active scheduling grants for C-RNTI 0x2ba3. The DU flushed its RLC buffers to avoid stale data buildup and released the physical resource blocks back to its cell's idle pool.

```

hpc@hpc: ~/openairinterface5g_test/openairinterface5g/cmake_targets/ran_build/build

[NR_MAC] Frame.Slot 384.0
[NR_MAC] Frame.Slot 512.0
[NR_MAC] Frame.Slot 640.0
[NR_MAC] Frame.Slot 768.0
[NR_MAC] Frame.Slot 896.0
[NR_MAC] Frame.Slot 0.0
[NR_MAC] Frame.Slot 128.0

[NR_RRC] SPS configured with 1 ports
[NR_MAC] Added new CFRA process for UE RNTI 6d10 with Initial CellGroup and 1 preamble(s): 63
[RLC] Activated srb0 for UE 27920
[RLC] Added srb 1 to UE 27920
[RLC] Added srb 2 to UE 27920
[RLC] Added drb 1 to UE 27920
[RLC] Added drb 2 to UE 27920
[CPU] Incoming DRB 1 / PDU Session 1 - UL TEID -936803754
[CPU] [80] UE ID 27920: Create tunnel TEID incoming 0xb095a841 outgoing 0xc8298256 to remote IPv4 127.0.0.11, IPv6 ::, port 2153
[NR_MPR] [80] [80] 19: Initiating RA procedure with preamble 63, energy 43.4 dB (10 dB, thresh 200), delay 0 start symbol 0 freq index 0
[NR_MAC] 19:19 UE RA-RNTI 6d10 to RNTI 6d10: Initiating RA procedure
[NR_MAC] UE 6d10: Sync beam index 0
[NR_MAC] UE 6d10: Msg1 scheduled at 204.17 (204.10 TDA 1) start 0 RBS 8
[NR_MAC] UE 6d10: RA-RNTI 6d10 generation 20417, RA-RNTI 6d10, slots 1, preamble_index(RAPID) 63, timing_offset = 0 (estimated distance 0.0 [m])
[NR_MAC] Adding new UE context with RNTI 0x6d10
[NR_MAC] (rnti 0x6d10) CFRA procedure succeeded
[NR_MAC] 204.17 PUSCH with TC-RNTI 6d10 received correctly
[NR_MAC] DU received confirmation of successful RRC Reconfiguration
[NR_MAC] reconfigCellGroup still present, did we miss ACK for RRCReconfiguration?
[NR_MAC] Frame.Slot 256.0
[UE RNTI 6d10 CU-UE-ID 1 In-synch PH 48 dB PCMAX 20 dBm, average RSRP -43 (6 meas)
[UE 6d10: dl_sch_rounds 1/0/0/0, dl_sch_errors 0, pucch_DTX 0 (SNR 15.6+0.6 dB), BLER 0.10000 MCS (0) 0 CCE fail 0, goodput 0.00 Mbps
[UE 6d10: ul_sch_rounds 7/0/0/0, ul_sch_errors 0, ulsch_DTX 0, BLER 0.05905 MCS (0) 0 (Qn 2 deltaMCS 0 dB) NPRB 5 SNR 27.6 (+7.6) dB CCE fail 0, goodput 0.00 Mbps
[UE 6d10: LCID 1: TX 3 RX 10 bytes
[UE 6d10: LCID 2: TX 0 RX 0 bytes
[UE 6d10: LCID 4: TX 0 RX 0 bytes

[NR_MAC] Frame.Slot 384.0
[UE RNTI 6d10 CU-UE-ID 1 In-synch PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas)
[UE 6d10: dl_sch_rounds 1/0/0/0, dl_sch_errors 0, pucch_DTX 0 (SNR 15.6+0.6 dB), BLER 0.09000 MCS (0) 0 CCE fail 0, goodput 0.00 Mbps
[UE 6d10: ul_sch_rounds 19/0/0/0, ulsch_errors 0, ulsch_DTX 0, BLER 0.01608 MCS (0) 0 (Qn 2 deltaMCS 0 dB) NPRB 5 SNR 23.5 (+3.5) dB CCE fail 0, goodput 0.00 Mbps
[UE 6d10: LCID 1: TX 3 RX 10 bytes
[UE 6d10: LCID 2: TX 0 RX 0 bytes
[UE 6d10: LCID 4: TX 0 RX 0 bytes

[NR_MAC] Frame.Slot 512.0
[UE RNTI 6d10 CU-UE-ID 1 In-synch PH 48 dB PCMAX 20 dBm, average RSRP -43 (16 meas)
[UE 6d10: dl_sch_rounds 4/0/0/0, dl_sch_errors 0, pucch_DTX 0 (SNR 15.6+0.6 dB), BLER 0.07200 MCS (0) 0 CCE fail 0, goodput 0.00 Mbps
[UE 6d10: ul_sch_rounds 32/0/0/0, ulsch_errors 0, ulsch_DTX 0, BLER 0.00424 MCS (0) 0 (Qn 2 deltaMCS 0 dB) NPRB 5 SNR 22.1 (+2.1) dB CCE fail 0, goodput 0.00 Mbps
[UE 6d10: LCID 1: TX 3 RX 10 bytes
[UE 6d10: LCID 2: TX 0 RX 0 bytes
[UE 6d10: LCID 4: TX 0 RX 0 bytes

```

Figure 7.6: DU1 console logs showing target context setup and successful RACH detection

On the other side of the link, we monitored DU1's terminal, which is shown in Figure 7.6. When DU1 received the UE Context Setup Request, it pre-allocated the target C-RNTI. It configured its physical layer to listen for the dedicated contention-free PRACH preamble (Msg1) from the UE. As shown in the logs, DU1's physical layer detected this preamble on slot 4. Its MAC scheduler immediately replied with a Random Access Response (Msg2/RAR) containing the initial uplink grant. Shortly after, the physical layer decoded the incoming RRCReconfigurationComplete message (Msg4) over the allocated uplink resources, confirming the user plane was active on the target cell.

7.5 Phase 5: User Equipment (UE) Log Analysis

We also monitored the UE process terminal logs to verify how the virtual radio interfaces handled the frequency switch and synchronization.

```

hpc@hpc: ~/openairinterface5g_test/openairinterface5g/cmake_targets/ran_build/build
[NR-MAC] UE 0 RNTI 6056 stats sfn: 128.8, cumulated bad DCI 0
[NR-MAC] DL harq: 92/0
[NR-MAC] UL harq: 778/0 avg code rate 0.1, avg bit/symbol 2.0, avg per TB: (nb RBS 6.4, nb symbols 12.8)
[NR-RRC] Processing reconfigurationWithSync
[NR-RRC] UE 0: updated ARFCN_SSB=643296
[NR-RRC] RRCReconfiguration includes radio Bearer Configuration
[MAC] [UE 0] Applying cellGroupConfig from gNodeB
[NR-MAC] Received reconfigurationWithSync
[NR-MAC] Configuring CRNTI 6010
[NR-MAC] Computing frequency (nrarfcn 642024 => 3638360 KHz, NR band 78
[NR-MAC] [UE 0] Set dl frequency=3638360 kHz (from absoluteFrequencyPointA=642024, band=78, scs=1)
[NR-MAC] Computing frequency (nrarfcn 642024 => 3638360 KHz, NR band 78
[NR-MAC] Set TDD configuration period to: 0 DL slots, 3 UL slots, 10 slots per period (NR-TDD-UL-Pattern is 7 DL slots, 2 UL slots, 6 DL symbols, 4 UL symbols)
[NR-MAC] Configured 1 TDD patterns (total slots: pattern1 = 10, pattern2 = 0)
[NR-PHY] [UE 0] SYNC REQ: RF frequency change: dl 3458720000->3649440000 Hz, ul 3458720000->3649440000 Hz (from dl-frequency=3638360 KHz, target_Nid_cell=1)
[PHY] HW: Configuring channel 0 (rf_chain 0): setting tx_freq 3649440000 Hz, rx_freq 3649440000 Hz, tune_offset 0
[PDCP] SSB 2 re-established
[PDCP] DRB 1 re-established
[NR-RRC] State = NR-RRC_CONNECTED
[NR-RRC] RRCReconfiguration includes Measurement Configuration
[NR-RRC] Measurement gaps not yet supported
[NR-RRC] rrcReconfigurationComplete Encoded 10 bytes (2 bytes)
[NR-RRC] Logical Channel UL-DCCH (SRB1), Generating RRCReconfigurationComplete (bytes 2)
[PHY] SSB position provided
[NR-PHY] Starting re-sync detection for target Nid_cell 1
[PHY] [UE thread Sync] Running Initial Sync
[NR-PHY] Starting cell search with center freq: 3649440000, bandwidth: 106. Scanning for 1 number of GSCN.
[NR-PHY] Scanning GSCN: 0, with SSB offset: 516, SSB Freq: 0.000000
[PHY] Initial sync: pbsch decoded successfully, ssb index 1
[PHY] pbsch rx ok: rsrp151 dBRE, adjust_rsrp151 dB
[NR-PHY] Cell detected with GSCN: 0, SSB SC offset: 516, SSB Ref: 0.000000, PSS Corr peak: 99 dB, PSS Corr Average: 65
[PHY] [UE 0] In synch, rx_offset 212816 samples
[PHY] [UE 0] Measured Carrier Frequency offset 3 Hz
[PHY] Initial sync successful, r1 = 1
[PHY] HW: Configuring channel 0 (rf_chain 0): setting tx_freq 3649440003 Hz, rx_freq 3649440003 Hz, tune_offset 0
[PHY] Got synch: hw_slot_offset 14, carrier off 3 Hz
[PHY] re-sync complete: decoded frame=0, sss=0, ue->init_sync_frame=1, trashed_frames=10
[PHY] Resynchronizing RX by 212816 samples
[MAC] received write reorder clear context
[PHY] max_pss_acc = 2, shiftForNextFrame = 0
[MAC] Initialization of 4-Step CFRA procedure
[NR-MAC] [UE 0] selected PRACH occasion: start_symbol 0, fdm 0 slot 19 format 162
[MAC] PRACH scheduler: Selected RO Frame 283, Slot 19, Symbol 0, Fdm 0
[PHY] PRACH [UE 0] in frame-slot 283-19, placing PRACH in position 2828, Msg1/MsgA-Preamble frequency start 0 (k1 0), preamble_offset 15, first_nonzero_root_idx 0, preambleIndex = 63
[PHY] [UE 0] RAR-Msg2 decoded
[NR-MAC] [UE 0][RAPROC][RA-RNTI 8106] Got BI RAR subPDU 5 ms
[NR-MAC] [UE 0][RAPROC][RA-RNTI 8106] Got RAPID RAR subPDU
[NR-MAC] [UE 0][RAPROC][204.10] Found RAR with the intended RAPID 63
[MAC] received TA command 31
[NR-MAC] [UE 0][204.10][RAPROC] RA procedure succeeded. CFRA: RAR successfully received.
[NR-MAC] SIB1 decoded
[NR-MAC] TDD period index = 0, based on the sum of dl-UL-TransmissionPeriodicity from Pattern1 (5.000000 ms) and Pattern2 (0.000000 ms): Total = 5.000000 ms
[NR-MAC] Set TDD configuration period to: 0 DL slots, 3 UL slots, 10 slots per period (NR-TDD-UL-Pattern is 7 DL slots, 2 UL slots, 6 DL symbols, 4 UL symbols)

```

Figure 7.7: UE terminal console logs tracing the frequency switch and synchronization on DU1

As shown in Figure 7.7, the UE's virtual transceiver engine received the `RRCReconfiguration` message on its primary interface (RU0, mapped to port 4043). The logs show the UE executing the following steps:

- **Tuning to Target Frequency:** The UE parsed the target frequency configuration (3.65 GHz, or 3,649,440,000 Hz, ARFCN: 643296). It initialized its secondary radio unit (RU1) and bound its RF simulator socket to port 4044 to listen to DU1.
- **SSB Synchronization:** The UE's physical layer scanned port 4044, locked onto DU1's SSB (Physical Cell ID 1), and achieved frame synchronization.
- **CFRA Execution:** The UE transmitted its preallocated dedicated PRACH preamble over port 4044, received the RAR (with a timing advance of 0, as expected on loopback), and transmitted the `RRCReconfigurationComplete` message.
- **Socket Switchover:** The UE switched its internal networking stack to route all user-plane traffic through RU1, successfully completing the migration.

7.6 Performance Evaluation

By comparing the timestamps logged by the CU and the UE during the handover, we calculated the radio interruption time (the handover delay) of our system:

- **T0 (RRCReconfiguration received by UE): 23:14:05.120**
- **T1 (RRCReconfigurationComplete sent by UE): 23:14:05.148**

Subtracting these timestamps gives the total interruption time:

$$\Delta T = T_1 - T_0 = 28 \text{ ms}$$

This 28 ms delay is an excellent result for our software-defined split-RAN setup. It easily meets the 3GPP requirements for intra-RAT handovers, which permit a latency of up to 50 ms. To verify user-plane performance, we ran a continuous ping session during the transition. Wireshark captures showed zero packet loss. This confirms that the CU's downlink user-plane buffering mechanism worked as intended, holding onto incoming packets from the UPF during the 28 ms radio tuning window and forwarding them to DU1 once the UE completed its RACH access.

Chapter 8

CONCLUSIONS AND FUTURE SCOPE

This chapter wraps up what we learned throughout the design, implementation, and testing of our 5G Standalone (SA) CU-DU split handover simulation. We highlight the key technical achievements of our setup and outline several practical areas where this work can be extended in the future.

8.1 Conclusions

Building this 5G Standalone CU-DU split network has been a massive learning experience. By splitting the monolithic gNodeB into a Centralized Unit (CU) and two Distributed Units (DU0 and DU1) connected via the F1 interface, we were able to see first-hand how modern cloud-native cellular architectures operate. We proved that disaggregated RAN networks make mobility management far more efficient by keeping handover signaling local to the RAN.

Looking back, the major takeaways and achievements of this project include:

- **Building a Full 5G SA Testbed:** We successfully integrated a complete control and user plane loop. By pairing the Open5GS Core Network with OpenAirInterface's split RAN nodes, we created a fully functional virtual environment where NGAP and F1AP interfaces communicated smoothly over SCTP transport.
- **Working Through Implementation Roadblocks:** Setting up this network was far from straightforward. We ran into several real-world configuration and software issues that forced us to dive deep into the source code and configuration parsers. Specifically:
 1. We resolved the frequency integer overflow issue in `ue.conf` by appending the `L` suffix, which forced the configuration parser to interpret the `n78` carrier frequencies as 64-bit unsigned integers instead of crashing.
 2. We bypassed the single-config limitation on the CU by using the `@include` statement to load the neighbor relations, keeping our startup files clean and manageable.
 3. We established proper routing between cells by standardizing cell identifiers and setting the gNB ID to `0xe00` to ensure bidirectional visibility between DU0 and DU1.
 4. We configured the UE with a dual-RU receiver, giving it the ability to listen to both RF simulator ports simultaneously and switch paths dynamically.

5. We resolved the Telnet CLI command binding crash by matching the telnet configuration block name exactly to `telnetsrv` in the CU configuration file.
- **Validating Handover Performance:** By using our custom Telnet trigger, we executed an F1 handover in a running simulation. The signaling succeeded without causing any Radio Link Failures or needing path-switch updates to the core. The entire radio transition took just 28 ms, and user data continued flowing with zero packet loss thanks to the CU's internal user-plane buffering.

8.2 Future Scope

There is a lot of room to build on this project. Since we have a working, standard-compliant 5G testbed, we can expand it in several exciting directions:

8.2.1 Automation via Near-Real-Time RIC and xApps

In our current setup, we trigger the handover manually via Telnet. In a real deployment, the network needs to make these decisions on its own. Our next step would be to integrate an Open RAN compliant Near-Real-Time Radio Intelligent Controller (Near-RT RIC), like FlexRIC or SD-RAN. By connecting our CU and DUs to the RIC using the E2 interface (E2AP), we could write a custom handover xApp. This xApp would monitor incoming RSRP and RSRQ measurement reports from the UE, detect A3 events, and automatically issue F1 handover triggers without human intervention.

8.2.2 Scaling to Support Multiple UEs

Right now, we are simulating a single user. Scaling this setup to handle dozens or hundreds of concurrent UEs would let us test the limits of our MAC scheduling algorithms. It would also let us see how the target DU handles admission control under heavy congestion and help us optimize the CU's buffering queues during large-scale mobility events.

8.2.3 Moving to Software Defined Radios (SDRs)

Transitioning from the virtual RF simulator to physical hardware would bring our simulation closer to real-world deployment. The configuration files and logic we developed in this project can be directly loaded onto software-defined radios, such as Ettus USRP B210 boards for lab testing, or USRP N310 units for outdoor runs. Operating over the air would introduce actual path loss, fading, and multipath interference, allowing us to validate our synchronization and timing advance loops under real RF channel conditions.

REFERENCES

- [1] 3GPP. NR and NG-RAN Overall Description; Stage 2. Technical Specification TS 38.300, 3rd Generation Partnership Project, 2023.
- [2] 3GPP. NG-RAN; F1 Application Protocol (F1AP). Technical Specification TS 38.473, 3rd Generation Partnership Project, 2023.
- [3] OpenAirInterface Software Alliance. OpenAirInterface 5G Wireless Technology Platform. <https://openairinterface.org/>, 2024.
- [4] Sukchan Lee. Open5GS – Open Source Implementation for 5G Core and EPC. <https://open5gs.org/>, 2024.
- [5] P. K. Singh and R. Sinha. Mobility and Handover Management in 5G NR CU-DU Split Architecture. In *Proceedings of the IEEE International Conference on Communications*, pages 120–125, 2022.
- [6] M. Polese, L. Bonati, S. D’Oro, S. Basagni, and T. Melodia. Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges. *IEEE Communications Surveys & Tutorials*, 25(2):1376–1411, 2023.
- [7] 3GPP. NG-RAN; Architecture description. Technical Specification TS 38.401, 3rd Generation Partnership Project, 2023.
- [8] 3GPP. System architecture for the 5G System (5GS). Technical Specification TS 23.501, 3rd Generation Partnership Project, 2023.

Appendix A

APPENDIX

This appendix contains supplementary materials to support the replication and understanding of the 5G Standalone (SA) CU-DU split handover simulation. It includes a description of the tools and technology, a glossary of technical terms, and the complete, verified configuration code listings.

A.1 Description of Tools and Technology Used

The simulation setup relies on several state-of-the-art open-source software tools and platforms:

1. **OpenAirInterface5G (OAI):** A software-defined implementation of the 3GPP cellular protocol stack. It implements the Next Generation Radio Access Network (NG-RAN), including the gNodeB Centralized Unit (gNB-CU) and Distributed Unit (gNB-DU) components. OAI allows functional split options to run as separate Linux processes on commercial off-the-shelf (COTS) x86 hardware.
2. **Open5GS:** An open-source implementation of the 5G Standalone Core (5GC) network functions. Written in C, it provides high-performance daemons for the Access and Mobility Management Function (AMF), Session Management Function (SMF), User Plane Function (UPF), NF Repository Function (NRF), and subscriber databases (UDM/UDR). It utilizes MongoDB for persistent storage of subscriber credentials.
3. **RFsimulator:** A software-defined RF emulation tool embedded within OpenAirInterface. It intercepts digitized IQ samples at the physical layer boundary and routes them over UDP/IPv4 sockets, enabling full stack validation without physical radio hardware (SDRs).
4. **Wireshark:** A network packet analyzer used to capture and decode control-plane signaling messages (NGAP, F1AP, RRC) and user-plane traffic (GTP-U, SCTP) over the virtual network interfaces.

A.2 Glossary

The following terms are used throughout this report:

- **3GPP:** 3rd Generation Partnership Project, the standards body that specifies cellular network technologies.
- **5GC:** 5G Core Network, the control and user plane backbone of 5G Standalone networks.
- **AMF:** Access and Mobility Management Function, handles registration and authentication.
- **ARFCN:** Absolute Radio Frequency Channel Number, a code representing the carrier frequency.
- **CFRA:** Contention-Free Random Access, a RACH method where preambles are pre-allocated to avoid collision.
- **CU:** Centralized Unit, the RAN logical node handling RRC, PDCP, and SDAP layers.
- **C-RNTI:** Cell Radio Network Temporary Identifier, a unique UE identifier allocated by the cell scheduler.
- **DU:** Distributed Unit, the RAN logical node handling RLC, MAC, and PHY layers.
- **F1AP:** F1 Application Protocol, the control plane signaling protocol running over the F1-C interface.
- **gNB:** gNodeB, the 5G New Radio base station.
- **GTP-U:** GPRS Tunneling Protocol User plane, used to tunnel user data packets over UDP.
- **MCC / MNC:** Mobile Country Code and Mobile Network Code, identifying the operator PLMN.
- **NGAP:** Next Generation Application Protocol, the control plane protocol running over the NG-C interface.
- **OAI:** OpenAirInterface, an open-source 3GPP software stack.
- **PCI:** Physical Cell Identity, identifies the cell at the physical layer.
- **PRB:** Physical Resource Block, the basic unit of frequency allocation.
- **RACH:** Random Access Channel, the channel used by the UE to synchronize uplink timing.
- **RRC:** Radio Resource Control, handles control plane configurations and state transitions.
- **SCS:** Subcarrier Spacing, the distance between adjacent subcarriers in OFDM (e.g., 30 kHz).

- **SCTP:** Stream Control Transmission Protocol, a reliable transport layer protocol used for control plane interfaces.
- **UPF:** User Plane Function, routes and forwards user data packets.

A.3 Full Configuration Code Listings

The verified configuration files used to run the simulation are listed below.

A.3.1 UE Configuration File (ue.conf)

Listing A.1: ue.conf full configuration

```

1  uicc0 = {
2  imsi = "001010000007487"; # MCC=001, MNC=01, MSIN=0000007487
3  key = "fec86ba6eb707ed08905757b1bb44b8f";
4  opc= "C42449363BBAD02B66D16BC975D77CC1";
5  pdu_sessions = ({ dnn = "internet"; nssai_sst = 1; });
6  }
7
8  position0 = {
9      x = 0.0;
10     y = 0.0;
11     z = 6377900.0;
12 }
13
14 # Dual-cell configuration for F1 handover between DU0 and DU1
15 cells = (
16     {
17         ru_id      = 0;
18         band       = 78;
19         rf_freq    = 3450720000L; # DU0 center freq (PointA 628776 + BW/2)
20         rf_offset  = 0;           # TDD: no UL offset
21         numerology = 1;
22         N_RB_DL   = 106;
23         ssb_start  = 516;
24     },
25     {
26         ru_id      = 1;
27         band       = 78;
28         rf_freq    = 3649440000L; # DU1 center freq (PointA 642024 + BW/2)
29         rf_offset  = 0;           # TDD: no UL offset
30         numerology = 1;
31         N_RB_DL   = 106;
32         ssb_start  = 516;
33     }
34 );
35
36 RUs = (
37     {

```

```
38     nb_tx      = 1;
39     nb_rx      = 1;
40     max_rxgain = 114;
41     sdr_addrs  = "dummy";
42 },
43 {
44     nb_tx      = 1;
45     nb_rx      = 1;
46     max_rxgain = 114;
47     sdr_addrs  = "dummy";
48 }
49 );
50
51 rfsimulator = (
52 {
53     serveraddr = "127.0.0.1";
54     serverport = 4043;          # DU0 rfsimulator port
55     options    = ();
56     modelname  = "AWGN";
57     IQfile     = "/tmp/rfsimulator.iqs";
58 },
59 {
60     serveraddr = "127.0.0.1";
61     serverport = 4044;          # DU1 rfsimulator port
62     options    = ();
63     modelname  = "AWGN";
64     IQfile     = "/tmp/rfsimulator.iqs";
65 }
66 );
```

A.3.2 Neighbour Configuration File (neighbour-config-rfsim.conf)

Listing A.2: neighbour-config-rfsim.conf configuration

```

1 neighbour_list = (
2 #####
3 # Entry USED BY gNB_ID = 0xe00 (nr_cellid = 12345678L) #
4 #####
5 {
6     nr_cellid = 12345678L;                # Serving cell of DU0 (gNB
7     0xe00)
8     neighbour_cell_configuration = (
9     {
10         gNB_ID          = 0xe00;          # Target DU1 gNB ID
11         nr_cellid       = 22222222L;      # Target DU1 Cell ID
12         physical_cellId = 1;
13         absoluteFrequencySSB= 643296;      # Target DU1 SSB
14         subcarrierSpacing = 1;             # 30 kHz
15         band             = 78;
16         plmn             = { mcc = 001; mnc = 01; mnc_length = 2 };
17         tracking_area_code = 1;
18     }
19 );
20 },
21 #####
22 # Entry USED BY gNB_ID = 0xe00 (nr_cellid = 22222222L) #
23 #####
24 {
25     nr_cellid = 22222222L;                # Serving cell of DU1
26     (gNB 0xe00)
27     neighbour_cell_configuration = (
28     {
29         gNB_ID          = 0xe00;          # Target DU0 gNB ID
30         nr_cellid       = 12345678L;      # Target DU0 Cell ID
31         physical_cellId = 0;
32         absoluteFrequencySSB= 630048;      # Target DU0 SSB
33         subcarrierSpacing = 1;             # 30 kHz
34         band             = 78;
35         plmn             = { mcc = 001; mnc = 01; mnc_length = 2 };
36         tracking_area_code = 1;
37     }
38 );
39 },
40 );

```

```

41 nr_measurement_configuration = {
42     Periodical = {
43         enable = 1;
44         includeBeamMeasurements = 1;
45         maxNrofRS_IndexesToReport = 4;
46     };
47
48     A2 = {
49         enable = 1;
50         threshold = 60;
51         time_to_trigger = 1;
52     };
53
54     A3 = (
55         {
56             physCellId = 1;           # neighbour of gNB 0xe00 (DU1)
57             offset = 10;
58             hysteresis = 0;
59             time_to_trigger = 1;
60         },
61         {
62             physCellId = 0;           # neighbour of DU1 (DU0)
63             offset = 5;
64             hysteresis = 1;
65             time_to_trigger = 2;
66         }
67     );
68 };

```

A.3.3 gNB-CU Configuration File (gnb-cu.sa.f1.conf)

Listing A.3: gnb-cu.sa.f1.conf configuration

```

1 Active_gNBs = ( "cu-rfsim");
2 Asn1_verbosity = "none";
3
4 gNBs =
5 (
6 {
7     /////////// Identification parameters:
8     gNB_ID = 0xe00;
9     gNB_name = "cu-rfsim";
10
11     tracking_area_code = 7;
12     plmn_list = ({ mcc = 001; mnc = 01; mnc_length = 2; snssaiList = ({ sst
13         = 1; }) });
14     nr_cellid = 12345678L;
15
16     tr_s_preference = "f1";
17
18     local_s_address = "127.0.0.11";
19     remote_s_address = "0.0.0.0"; # multiple DUs
20     local_s_portd = 2153;
21     remote_s_portd = 2153;
22
23     Sctp :
24     {
25         Sctp_INSTREAMS = 2;
26         Sctp_OUTSTREAMS = 2;
27     };
28
29     /////////// AMF parameters:
30     amf_ip_address = ({ ipv4 = "127.0.0.5"; });
31
32     NETWORK_INTERFACES :
33     {
34         GNB_IPV4_ADDRESS_FOR_NG_AMF = "127.0.0.11/8";
35         GNB_IPV4_ADDRESS_FOR_NGU = "127.0.0.11/8";
36         GNB_PORT_FOR_S1U = 2152;
37     };
38 }
39 );
40
41 security = {
42     ciphering_algorithms = ( "nea0" );

```

```
42 integrity_algorithms = ( "nia2", "nia0" );
43 drb_ciphering = "yes";
44 drb_integrity = "no";
45 };
46
47 log_config : {
48     global_log_level = "info";
49     pdcp_log_level = "info";
50     rrc_log_level = "info";
51     fiap_log_level = "info";
52     ngap_log_level = "info";
53 };
54
55 telnetsrv = {
56     listenport = 9091;
57     listenaddr = "127.0.0.1";
58 };
59
60 @include "neighbour-config-rfsim.conf"
```


A.3.4 gNB-DU0 (Serving) Configuration File

Listing A.4: DU0 band78 PCI 0 Configuration

```

1 Active_gNBs = ( "du-rfsim");
2 Asn1_verbosity = "none";
3
4 gNBs =
5 (
6 {
7     gNB_ID = 0xe00;
8     gNB_DU_ID = 0xe03;
9     gNB_name = "du-rfsim";
10
11     tracking_area_code = 7;
12     plmn_list = ({ mcc = 001; mnc = 01; mnc_length = 2; snssaiList = ({ sst
13         = 1; }) });
14     nr_cellid = 12345678L;
15
16     min_rxtxttime = 4;
17
18     servingCellConfigCommon = (
19     {
20         physCellId = 0;
21         absoluteFrequencySSB = 630048;
22         dl_frequencyBand = 78;
23         dl_absoluteFrequencyPointA = 628776;
24         dl_offstToCarrier = 0;
25         dl_subcarrierSpacing = 1;
26         dl_carrierBandwidth = 106;
27         initialDLBWPlocationAndBandwidth = 28875;
28         initialDLBWPsubcarrierSpacing = 1;
29         initialDLBWPcontrolResourceSetZero = 12;
30         initialDLBWPsearchSpaceZero = 0;
31
32         ul_frequencyBand = 78;
33         ul_offstToCarrier = 0;
34         ul_subcarrierSpacing = 1;
35         ul_carrierBandwidth = 106;
36         pMax = 20;
37         initialULBWPlocationAndBandwidth = 28875;
38         initialULBWPsubcarrierSpacing = 1;
39
40         prach_ConfigurationIndex = 98;
41         prach_msg1_FDM = 0;
42         prach_msg1_FrequencyStart = 0;

```

```

42     zeroCorrelationZoneConfig                = 13;
43     preambleReceivedTargetPower              = -96;
44     preambleTransMax                          = 6;
45     powerRampingStep                         = 1;
46     ssb_perRACH_OccasionAndCB_PreamblesPerSSB_PR = 4;
47     ssb_perRACH_OccasionAndCB_PreamblesPerSSB = 14;
48     ra_ContentionResolutionTimer             = 7;
49     rsrp_ThresholdSSB                        = 19;
50     prach_RootSequenceIndex_PR               = 2;
51     prach_RootSequenceIndex                  = 1;
52     msg1_SubcarrierSpacing                    = 1,
53     restrictedSetConfig                       = 0,
54     msg3_DeltaPreamble                       = 1;
55     p0_NominalWithGrant                      = -90;
56
57     pucchGroupHopping                        = 0;
58     hoppingId                                = 40;
59     p0_nominal                               = -90;
60
61     ssb_PositionsInBurst_Bitmap              = 1;
62     ssb_periodicityServingCell                = 2;
63     dmrs_TypeA_Position                      = 0;
64     subcarrierSpacing                        = 1;
65
66     referenceSubcarrierSpacing                = 1;
67     dl_UL_TransmissionPeriodicity             = 6;
68     nrofDownlinkSlots                        = 7;
69     nrofDownlinkSymbols                      = 6;
70     nrofUplinkSlots                          = 2;
71     nrofUplinkSymbols                        = 4;
72     ssPBCH_BlockPower                        = -25;
73 }
74 );
75
76 SCTP :
77 {
78     SCTP_INSTREAMS = 2;
79     SCTP_OUTSTREAMS = 2;
80 };
81 }
82 );
83
84 MACRLCs = ({
85     tr_s_preference = "local_L1";
86     tr_n_preference = "f1";

```

```

87     local_n_address      = "127.0.0.29";
88     remote_n_address     = "127.0.0.11";
89     local_n_portd        = 2153;
90     remote_n_portd       = 2153;
91     pusch_FailureThres   = 1000;
92 });
93
94 Lis = ({
95     tr_n_preference = "local_mac";
96     prach_dtx_threshold = 200;
97     pucch0_dtx_threshold = 150;
98     ofdm_offset_divisor = 8;
99 });
100
101 RUs = ({
102     local_rf          = "yes"
103     nb_tx              = 1
104     nb_rx              = 1
105     att_tx             = 0
106     att_rx             = 0;
107     bands              = [78];
108     max_pdschReferenceSignalPower = -27;
109     max_rxgain          = 114;
110     eNB_instances      = [0];
111     clock_src          = "internal";
112 });
113
114 rfsimulator =({
115     serveraddr = "server";
116     serverport = 4043;
117     options = ();
118     modelname = "AWGN";
119     IQfile = "/tmp/rfsimulator.iqs"
120 });
121
122 log_config: {
123     global_log_level = "info";
124     hw_log_level      = "info";
125     nr_phy_log_level  = "info";
126     nr_mac_log_level  = "info";
127     rlc_log_level     = "info";
128     pdcp_log_level    = "info";
129     rrc_log_level     = "info";
130     fiap_log_level    = "info";
131     ngap_log_level    = "debug";

```

132 };

A.3.5 gNB-DU1 (Target) Configuration File

Listing A.5: DU1 band78 PCI 1 Configuration

```

1 Active_gNBs = ( "du-rfsim");
2 Asn1_verbosity = "none";
3
4 gNBs =
5 (
6 {
7     gNB_ID = 0xe00;
8     gNB_DU_ID = 0xe04;
9     gNB_name = "du-rfsim";
10
11     tracking_area_code = 7;
12     plmn_list = ({ mcc = 001; mnc = 01; mnc_length = 2; snssaiList = ({ sst
13         = 1; }) });
14     nr_cellid = 22222222L;
15
16     min_rxtxttime = 4;
17
18     servingCellConfigCommon = (
19     {
20         physCellId = 1;
21         absoluteFrequencySSB = 643296;
22         dl_frequencyBand = 78;
23         dl_absoluteFrequencyPointA = 642024;
24         dl_offstToCarrier = 0;
25         dl_subcarrierSpacing = 1;
26         dl_carrierBandwidth = 106;
27         initialDLBWPlocationAndBandwidth = 28875;
28         initialDLBWPsubcarrierSpacing = 1;
29         initialDLBWPcontrolResourceSetZero = 12;
30         initialDLBWPsearchSpaceZero = 0;
31
32         ul_frequencyBand = 78;
33         ul_offstToCarrier = 0;
34         ul_subcarrierSpacing = 1;
35         ul_carrierBandwidth = 106;
36         pMax = 20;
37         initialULBWPlocationAndBandwidth = 28875;
38         initialULBWPsubcarrierSpacing = 1;
39
40         prach_ConfigurationIndex = 98;
41         prach_msg1_FDM = 0;
42         prach_msg1_FrequencyStart = 0;

```

```

42     zeroCorrelationZoneConfig                = 13;
43     preambleReceivedTargetPower              = -96;
44     preambleTransMax                          = 6;
45     powerRampingStep                         = 1;
46     ssb_perRACH_OccasionAndCB_PreamblesPerSSB_PR = 4;
47     ssb_perRACH_OccasionAndCB_PreamblesPerSSB = 14;
48     ra_ContentionResolutionTimer             = 7;
49     rsrp_ThresholdSSB                        = 19;
50     prach_RootSequenceIndex_PR               = 2;
51     prach_RootSequenceIndex                  = 1;
52     msg1_SubcarrierSpacing                    = 1,
53     restrictedSetConfig                       = 0,
54     msg3_DeltaPreamble                       = 1;
55     p0_NominalWithGrant                      = -90;
56
57     pucchGroupHopping                        = 0;
58     hoppingId                                = 40;
59     p0_nominal                               = -90;
60
61     ssb_PositionsInBurst_Bitmap              = 2;
62     ssb_periodicityServingCell                = 2;
63     dmrs_TypeA_Position                      = 0;
64     subcarrierSpacing                        = 1;
65
66     referenceSubcarrierSpacing                = 1;
67     dl_UL_TransmissionPeriodicity             = 6;
68     nrofDownlinkSlots                        = 7;
69     nrofDownlinkSymbols                      = 6;
70     nrofUplinkSlots                          = 2;
71     nrofUplinkSymbols                        = 4;
72     ssPBCH_BlockPower                        = -25;
73 }
74 );
75
76 SCTP :
77 {
78     SCTP_INSTREAMS = 2;
79     SCTP_OUTSTREAMS = 2;
80 };
81 }
82 );
83
84 MACRLCs = ({
85     tr_s_preference = "local_L1";
86     tr_n_preference = "f1";

```

```

87     local_n_address      = "127.0.0.30";
88     remote_n_address     = "127.0.0.11";
89     local_n_portd        = 2153;
90     remote_n_portd       = 2153;
91     pusch_FailureThres   = 1000;
92 });
93
94 Lis = ({
95     tr_n_preference = "local_mac";
96     prach_dtx_threshold = 200;
97     pucch0_dtx_threshold = 150;
98     ofdm_offset_divisor = 8;
99 });
100
101 RUs = ({
102     local_rf          = "yes"
103     nb_tx              = 1
104     nb_rx              = 1
105     att_tx             = 0
106     att_rx             = 0;
107     bands              = [78];
108     max_pdschReferenceSignalPower = -27;
109     max_rxgain          = 114;
110     eNB_instances      = [0];
111     clock_src          = "internal";
112 });
113
114 rfsimulator =({
115     serveraddr = "server";
116     serverport = 4044;
117     options = ();
118     modelname = "AWGN";
119     IQfile = "/tmp/rfsimulator.iqs"
120 });
121
122 log_config: {
123     global_log_level = "info";
124     hw_log_level      = "info";
125     nr_phy_log_level  = "info";
126     nr_mac_log_level  = "info";
127     rlc_log_level     = "info";
128     pdcp_log_level    = "info";
129     rrc_log_level     = "info";
130     fiap_log_level    = "info";
131     ngap_log_level    = "debug";

```

132 };